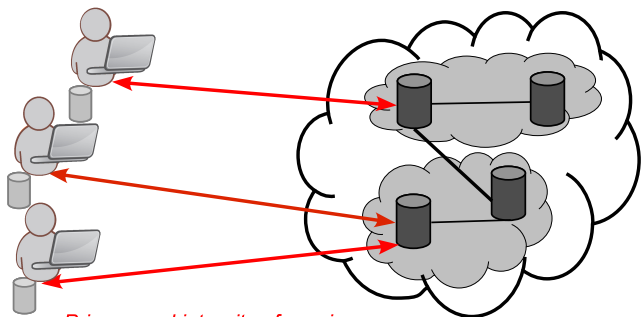


Privacy and Data Protection in Emerging Scenarios

Security, Privacy, and Data Protection Laboratory
Dipartimento di Informatica
Università degli Studi di Milano

Privacy and integrity of queries and computations



*Privacy and integrity of queries
and computations*

Privacy of users

Privacy and integrity of data storage

Access and pattern confidentiality

Guaranteeing privacy of outsourced data entails protecting the confidentiality of the data (**content confidentiality**) as well as of the **accesses to them**

- **Access confidentiality**: confidentiality of the fact that an access aims at a specific data
- **Pattern confidentiality**: confidentiality of the fact that two accesses aim at the same data

Approaches for protecting data accesses

- Private Information Retrieval (PIR) proposals (e.g., [CKGS-98, SC-07])
- Oblivious traversal of tree-structured data/indexes [LC-04]
- Pyramid-shaped database layout of Oblivious RAM [WSC-08, WS-12]
- Path ORAM protocol, working on a tree structure [SVSFRYD-13]
- Ring ORAM, variation of Path ORAM with better performance and same protection guarantees [RFKSSvD-15]
- Shuffle index based on the definition of a B+-tree structure with dynamic allocation of data [DFPPS-11a, DFPPS-11b, DFPPS-13]

Path ORAM

Server side

- Binary tree structure with L levels ($L = \lceil \log_2(N) \rceil - 1$, with N the number of blocks)
- Each node in the tree is a **bucket** that contains up to Z real blocks (padded with **dummy blocks**)
- Each leaf node x defines a unique path $P(x)$ from x to the root

Client side

- The client locally stores a small number of blocks in a **stash**
- The client stores a position map: $x = \text{position}[a]$ means that a block identified by a is currently mapped to the x -th leaf node $\implies$ block a (if it exists) resides in some bucket in path $P(x)$ or in the stash
- The position map changes every time blocks are accessed and remapped

Path ORAM – Main invariant

At any time:

- each block is mapped to a uniformly random leaf bucket in the tree
- unstashed blocks are always placed in some bucket along the path to the mapped leaf

Path ORAM reads and writes

1. **Remap block:** Let x be the old position of a . Randomly remap the position of a to a new random position (a new leaf node)
2. **Read path:** read nodes in $P(x)$ containing a .
If the access is a write, update the data stored for block a
3. **Write path:** write the nodes in $P(x)$ back possibly including some additional blocks from the stash if they can be placed into the path (i.e., the main invariant is satisfied)

Path ORAM – Example

Client

stash

a	b				
---	---	--	--	--	--

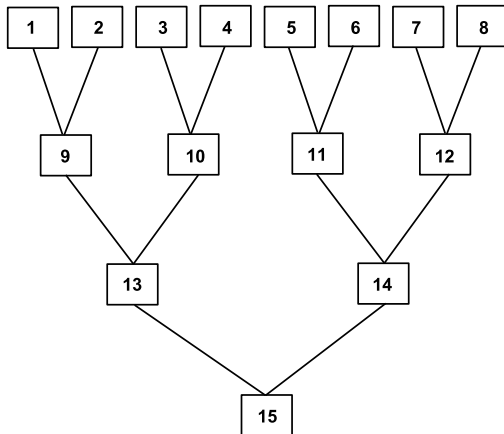
position[a] = 4

position[b] = 5

position[c] = 7

...

Server



Path ORAM – Example

Client

Read access to block: **c**

stash

a	b				
---	---	--	--	--	--

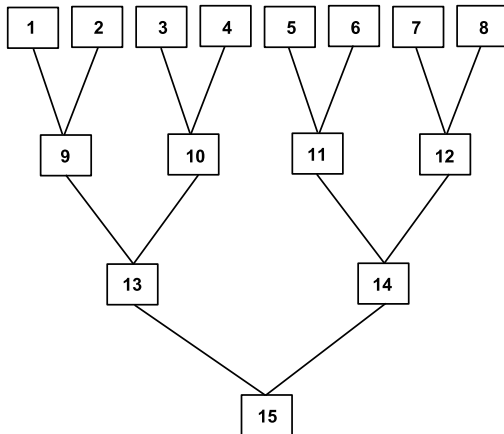
position[a] = 4

position[b] = 5

position[c] = 7

...

Server



Path ORAM – Example

Client

Read access to block: **c**

1. $x := \text{position}[c] = 7$

$\text{position}[c] := \text{Random}(1, \dots, 8) = 8$

stash

a	b				
---	---	--	--	--	--

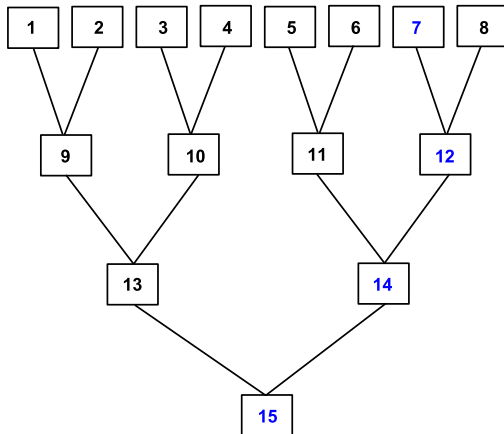
$\text{position}[a] = 4$

$\text{position}[b] = 5$

$\text{position}[c] = 8$

...

Server



Path ORAM – Example

Client

Read access to block: c

1. $x := \text{position}[c] = 7$

$\text{position}[c] := \text{Random}(1, \dots, 8) = 8$

2. Read path $P(7)$

stash



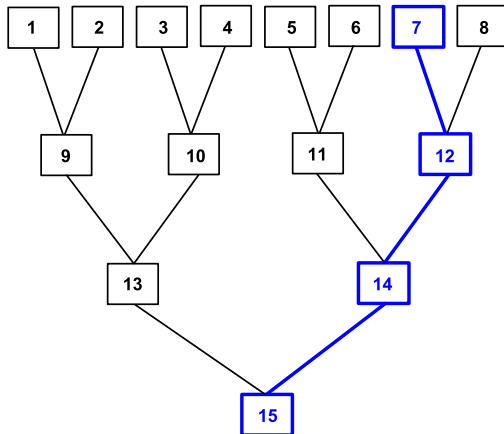
$\text{position}[a] = 4$

$\text{position}[b] = 5$

$\text{position}[c] = 8$

...

Server



Path ORAM – Example

Client

Read access to block: **c**

1. $x := \text{position}[c] = 7$

$\text{position}[c] := \text{Random}(1, \dots, 8) = 8$

2. Read path $P(7)$

3. Write back nodes in $P(7)$; move nodes whose path intersects $P(7)$ to the highest intersecting node (a not written back - the highest intersecting node is full
b written in node 14
c written in node 12)

stash

a					
---	--	--	--	--	--

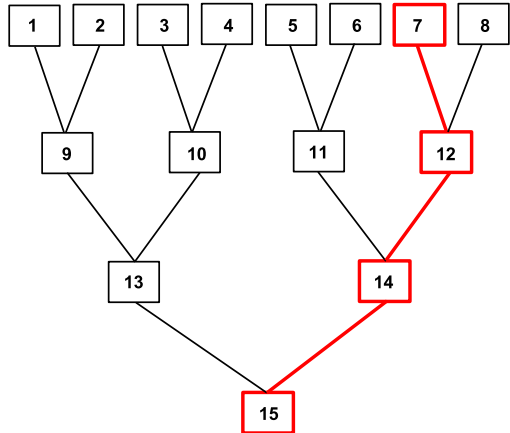
$\text{position}[a] = 4$

$\text{position}[b] = 5$

$\text{position}[c] = 8$

...

Server



Ring ORAM

- Variation of Path ORAM that reduces the online access bandwidth to $O(1)$ and the overall bandwidth to $\sim 2 - 2.5\log(N)$
- Same server-side structure as Path ORAM but each node has
 - S additional dummy blocks
 - a small map of the offsets of its blocks
 - a counter of accesses
- Protocol
 - Remap (step 1) is the same as Path ORAM
 - Read path (step 2) is revised to download only one block per bucket
 - Write path (step 3) is factorized among multiple access operations (eviction phase)

Path ORAM and Ring ORAM: Pros and cons

Path ORAM and Ring ORAM provide access and pattern confidentiality

- + same protection guarantees as ORAM (no inferences)
- + much more efficient than ORAM $\implies$ more applicable in practice
- + limited access time
- range queries are not supported
- accesses by multiple clients are not supported
- vulnerable to failures of the client
- $\sim 2 - 2.5 \log(N)$ overall bandwidth overhead w.r.t. non protected accesses

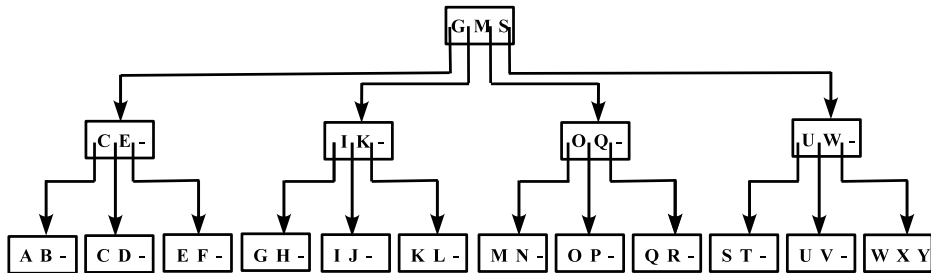
Shuffle Index

S. De Capitani di Vimercati, S. Foresti, S. Paraboschi, G. Pelosi, P. Samarati, "Efficient and Private Access to Outsourced Data," in *Proc. of ICDCS*, Minneapolis, MN, USA, June 2011.

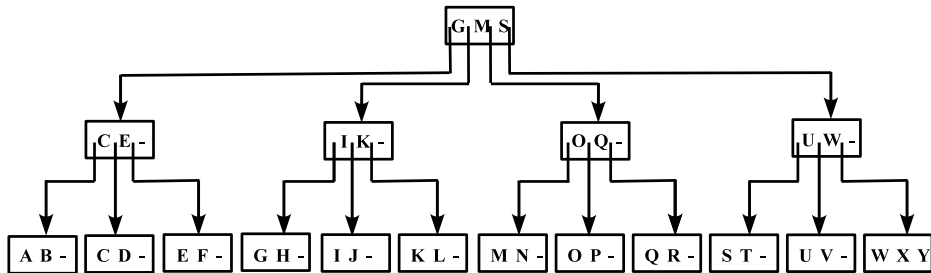
Shuffle index data structure

- Data are indexed over a candidate key K and organized as an **unchained $B+$ -tree** with fan out F
- Data are stored in the leaves in association with their index values
- Accesses to the data (searches) are based on the value of the index
- **Node structure:**
 - $q \geq \lceil F/2 \rceil$ children with $q - 1$ values $v_1 \leq \dots \leq v_{q-1}$
 - i -th child is the root of a subtree containing the values v with: $v < v_1$; $v_{i-1} \leq v < v_i$, $i = 2, \dots, q - 2$; $v \geq v_{q-1}$

Abstract representation of shuffle index – Example

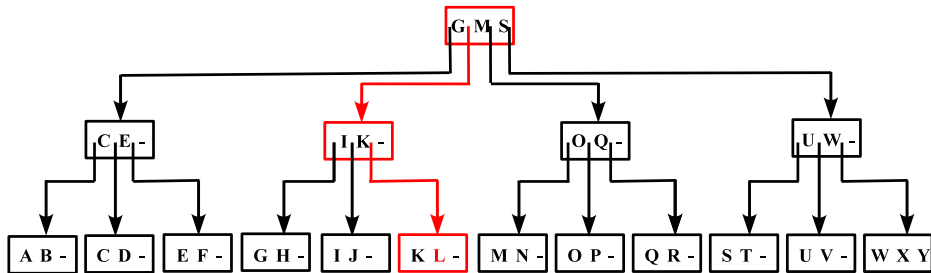


Abstract representation of shuffle index – Example



Search: L

Abstract representation of shuffle index – Example

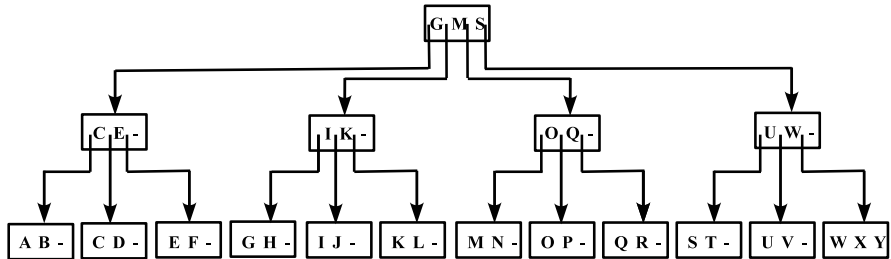


Search: L

Logical representation of shuffle index

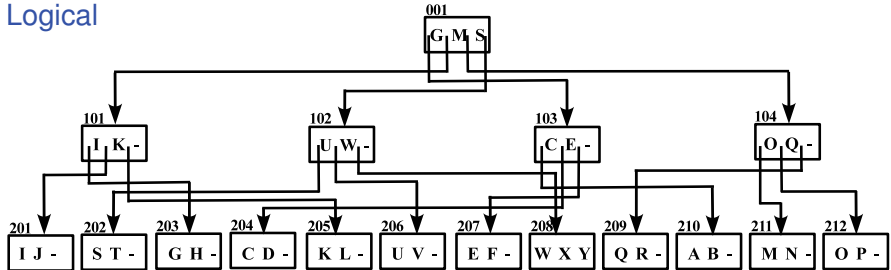
- Pointers between nodes of the abstract data structure correspond, at **logical level**, to node identifiers
- Set of pairs $\langle \text{id}, n \rangle$, with **id** the node identifier and **n** the node content
 - the order between identifiers does not necessarily correspond to the order in which nodes appear in the abstract representation

Abstract and logical shuffle index – Example



Abstract

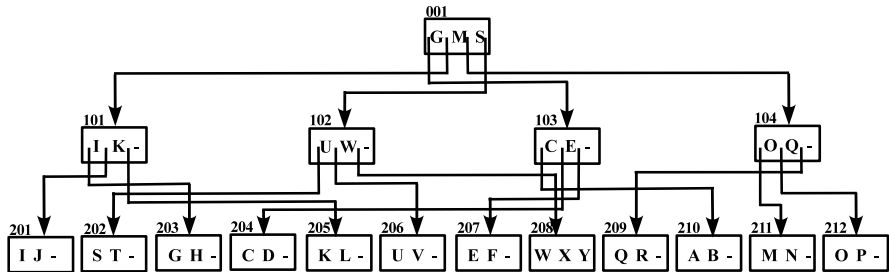
Logical



Physical representation of shuffle index

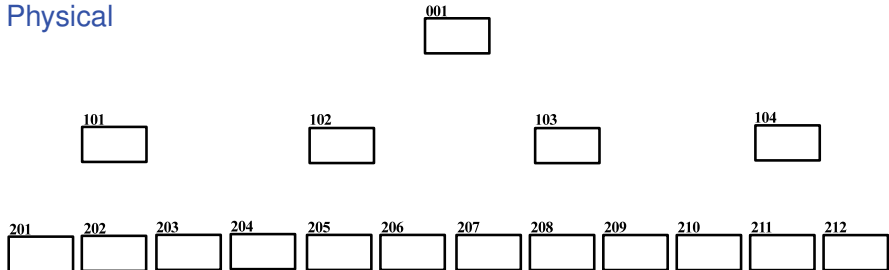
- Each node $\langle \text{id}, n \rangle$ of the logical shuffle index is stored on the server in encrypted form (content confidentiality)
- A node $\langle \text{id}, n \rangle$ corresponds to a block $\langle \text{id}, b \rangle$, with $b = \mathcal{C} || \mathcal{T}$, $\mathcal{C} = E_k(s || n)$, $\mathcal{T} = \text{MAC}_{k'}(\text{id} || \mathcal{C})$, s a value chosen at random during each encryption

Logical and physical shuffle index – Example



Logical

Physical

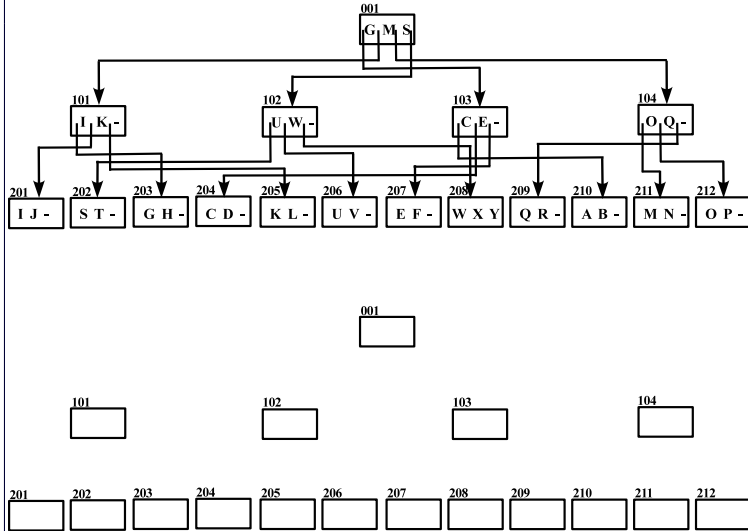


Data accesses

- Access to the data requires an **iterative process** between the **client** and the **server**
- The client performs an iteration for each level of the shuffle index starting from the root
- At each iteration, the client:
 - decrypts the retrieved block
 - determines the block to be retrieved from the server at the next level
- The process ends when a leaf block is retrieved

Data accesses – Example

Search: F

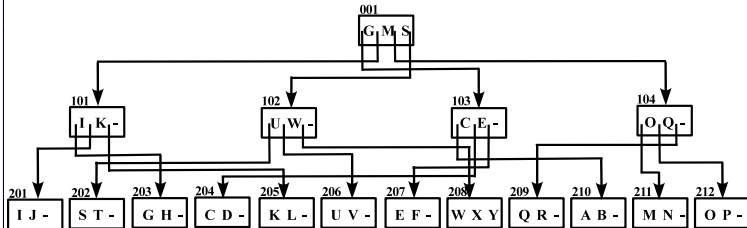


Data accesses – Example

Search: F

level: 0

download: 001



001

101

102

103

104

201

202

203

204

205

206

207

208

209

210

211

212

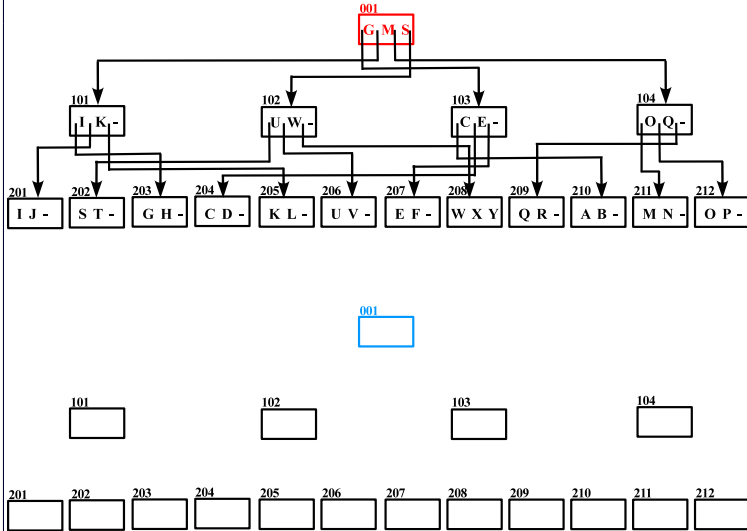
Data accesses – Example

Search: F

level: 0

download: 001

decrypt: 001



Data accesses – Example

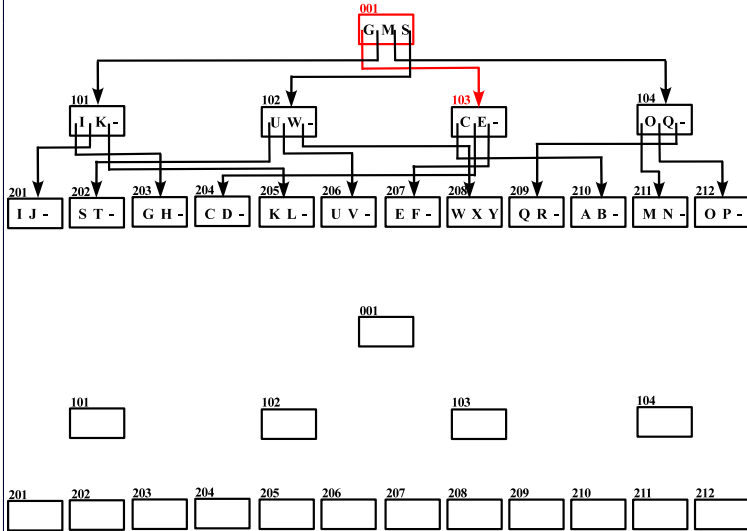
Search: F

level: 0

download: 001

decrypt: 001

level: 1



Data accesses – Example

Search: F

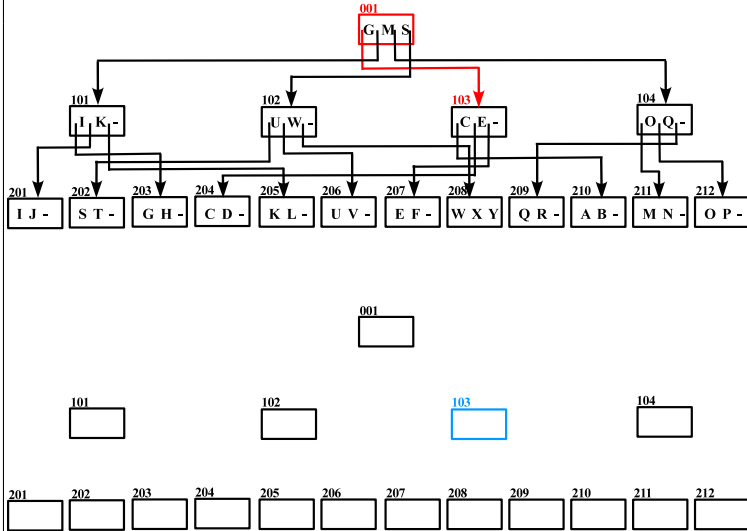
level: 0

download: 001

decrypt: 001

level: 1

download: 103



Data accesses – Example

Search: F

level: 0

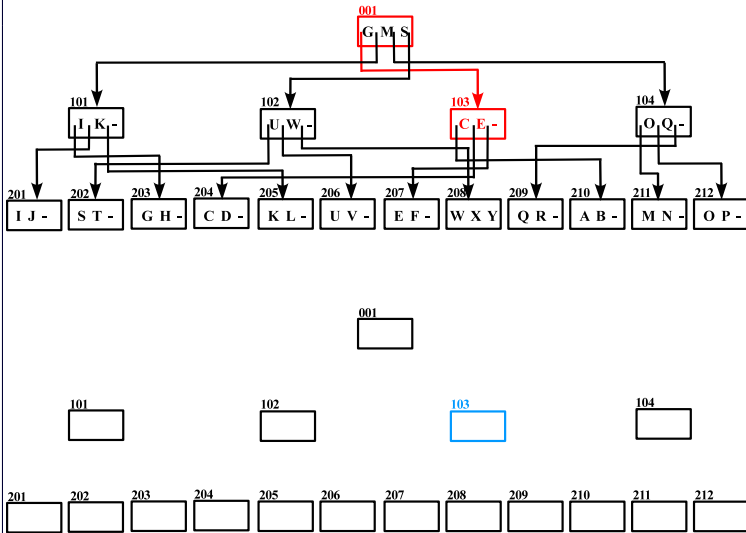
download: 001

decrypt: 001

level: 1

download: 103

decrypt: 103



Data accesses – Example

Search: F

level: 0

download: 001

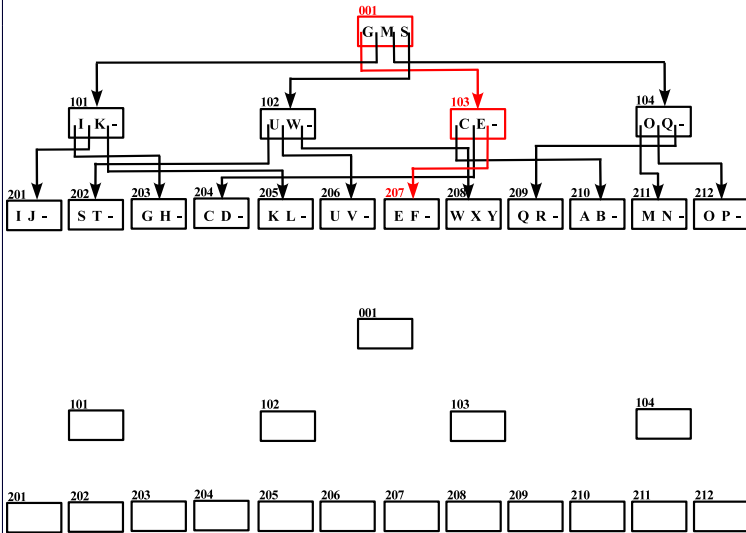
decrypt: 001

level: 1

download: 103

decrypt: 103

level: 2



Data accesses – Example

Search: F

level: 0

download: 001

decrypt: 001

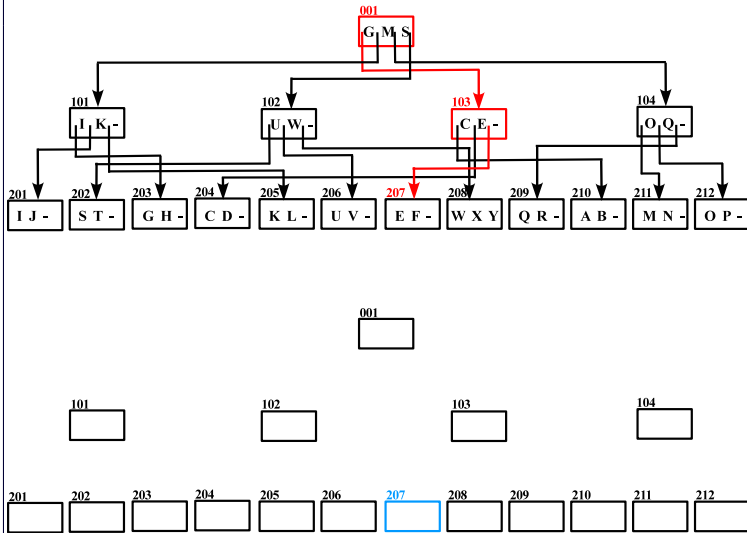
level: 1

download: 103

decrypt: 103

level: 2

download: 207



Data accesses – Example

Search: F

level: 0

download: 001

decrypt: 001

level: 1

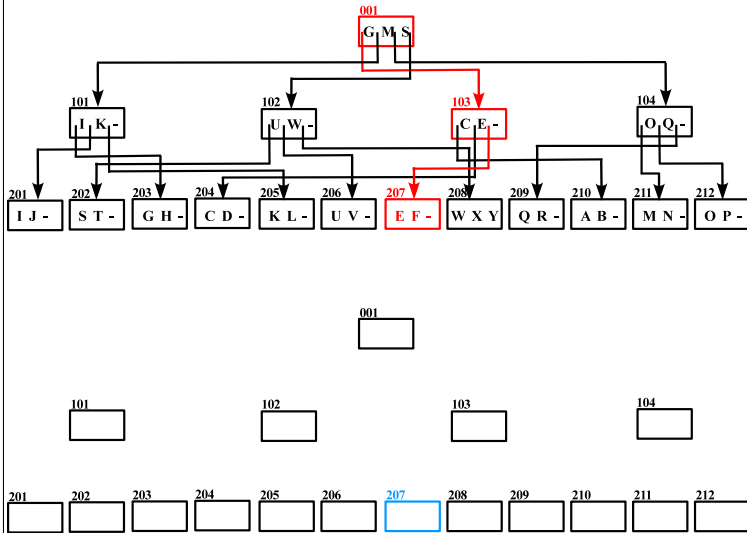
download: 103

decrypt: 103

level: 2

download: 207

decrypt: 207



Knowledge of the observer (server)

- The server receives a set of blocks to store
- The server receives requests to access the blocks that translate into **observations**
 - an observation o_i corresponds to a sequence of blocks $\{b_{i1}, \dots, b_{ih}\}$
- The server knows or can easily infer:
 - the number m of blocks and their identifiers
 - the height h of the shuffle index
 - the level associated with each block (after the observation of a long history of accesses)

Problem statement

Given a sequence of observations $\{o_1, \dots, o_z\}$ the server should not be able to infer:

- the data stored in the shuffle index (content confidentiality)
- the data to which access requests are aimed, that is, $\forall i = 1, \dots, z$, the server should not infer that o_i aims at a specific node (access confidentiality)
- o_i aims at accessing the same node as o_j , $\forall i, j = 1, \dots, z, i \neq j$ (pattern confidentiality)

Is encryption enough?

+ It protects:

- content confidentiality of data at rest
- access confidentiality of individual requests

— Access and pattern confidentiality is not provided

- accesses to the same blocks imply accesses to the same data

⇒ frequency-based attacks allow the server to reconstruct the correspondence between plaintext values and blocks

Rationale of the approach

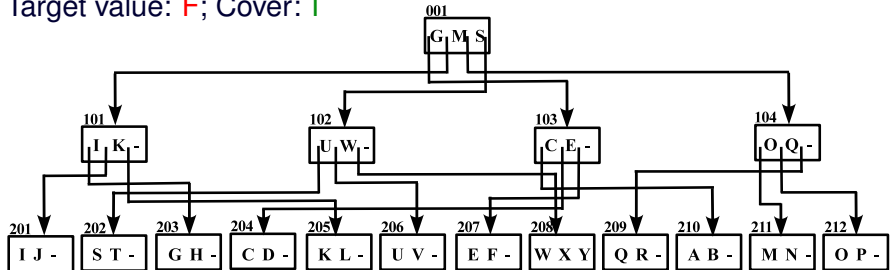
- Destroy the correspondence between the frequencies with which blocks are accessed and the frequencies of accesses to different values
- Combine three strategies:
 - cover searches
 - provide confusion in individual accesses
 - cached searches
 - allow protection of accesses to the same values
 - shuffling
 - dynamically changes node allocation to blocks at every access, so destroying the fixed node-block correspondence

Cover searches

- Introduce **confusion** on the target of an access by hiding it within a group of other requests that act as covers
- The number of covers (**num_cover**) is a protection parameter
- Cover searches must:
 - provide **block diversity** (i.e., on a path disjoint from the target searched, apart from the root)
 - be **indistinguishable** from actual searches (i.e., enjoy a believable frequency of access)

Cover searches – Example (1)

Target value: **F**; Cover: **I**



001
[]

101
[]

102
[]

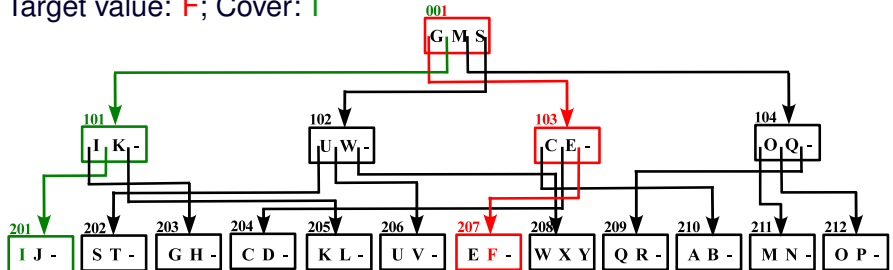
103
[]

104
[]

201 [] 202 [] 203 [] 204 [] 205 [] 206 [] 207 [] 208 [] 209 [] 210 [] 211 [] 212 []

Cover searches – Example (1)

Target value: **F**; Cover: **I**



001

101

102

103

104

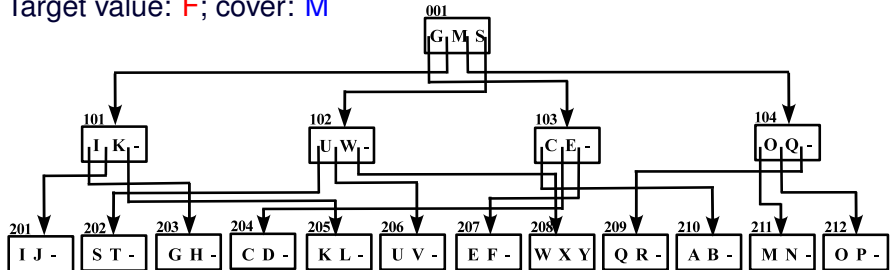
201	202	203	204	205	206	207	208	209	210	211	212

Cover searches – Protection offered

- + Leaf blocks have the same probability of containing the actual target
 - o e.g., blocks 201 and 207 can be both the target block
- + The parent-child relationship between accessed blocks is confused
 - o e.g., block 201 could be child of either 101 or 103
- Parent-child relationship can be disclosed by [intersection attacks](#)

Cover searches – Example (2)

Target value: **F**; cover: **M**



001
[]

101
[]

102
[]

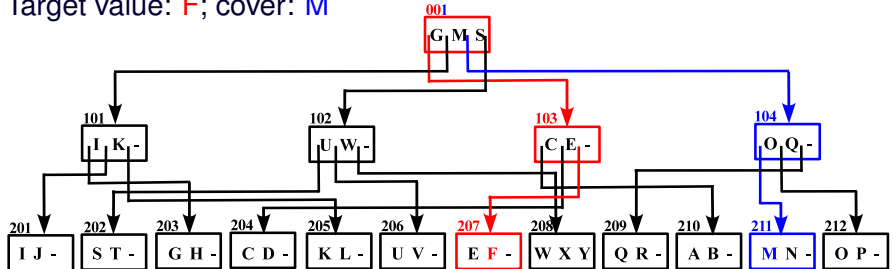
103
[]

104
[]

201 [] 202 [] 203 [] 204 [] 205 [] 206 [] 207 [] 208 [] 209 [] 210 [] 211 [] 212 []

Cover searches – Example (2)

Target value: **F**; cover: **M**



001
[]

101
[]

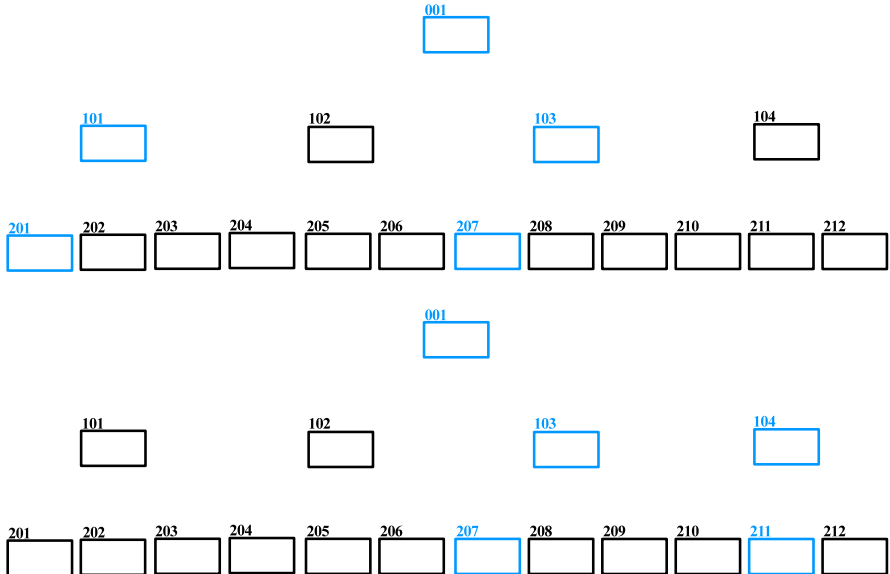
102
[]

103
[]

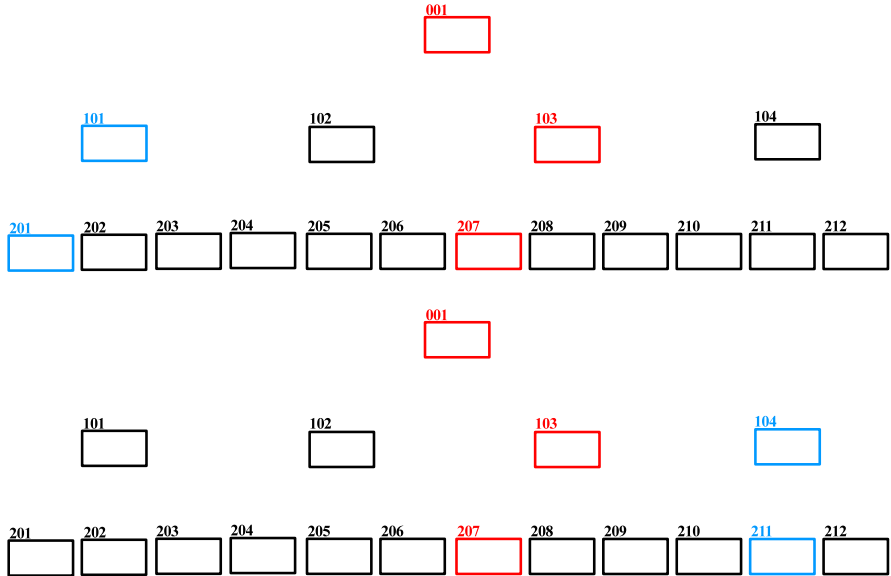
104
[]

201 [] 202 [] 203 [] 204 [] 205 [] 206 [] 207 [] 208 [] 209 [] 210 [] 211 [] 212 []

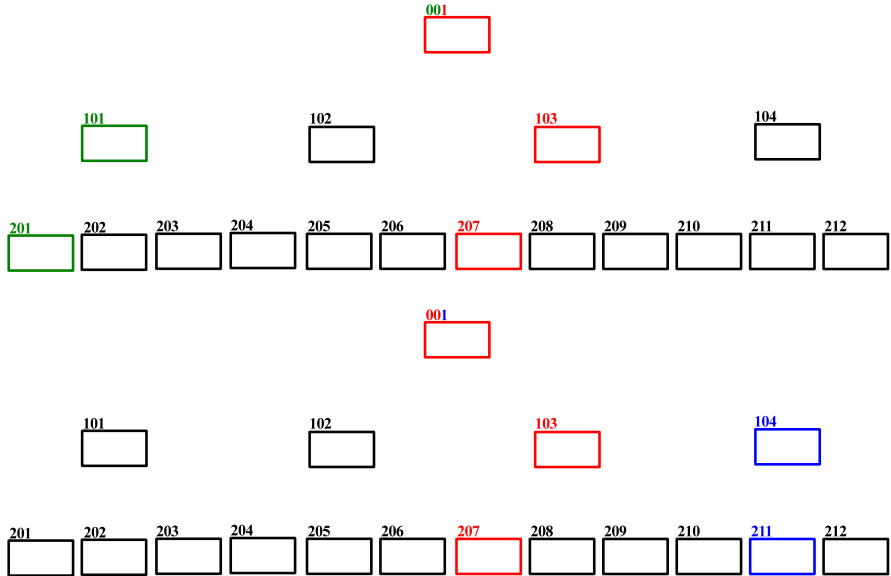
Cover searches – Intersection attack



Cover searches – Intersection attack



Cover searches – Intersection attack



Cached searches

- The client maintains a local **cache** of nodes in the path to the target for counteracting intersection attacks
 - initialized with **num_cache** disjoint paths and is managed according to the LRU policy
 - if a node is in cache, its parent also is (**path continuity property**)
 - refreshed at every access
 - recently searched nodes will be found in the cache
 - if a target node is in cache, only cover searches will be performed
 - provides fake observations for the server
 - allows (with shuffling) refreshing the cache

Cached searches – Example (1)

l | $Cache_l$

0 | 001 [$_{103}G_{101}M_{104}S_{102}$]

1 | 102 [$_{202}U_{206}W_{208}-$]

2 | 202 [ST-]

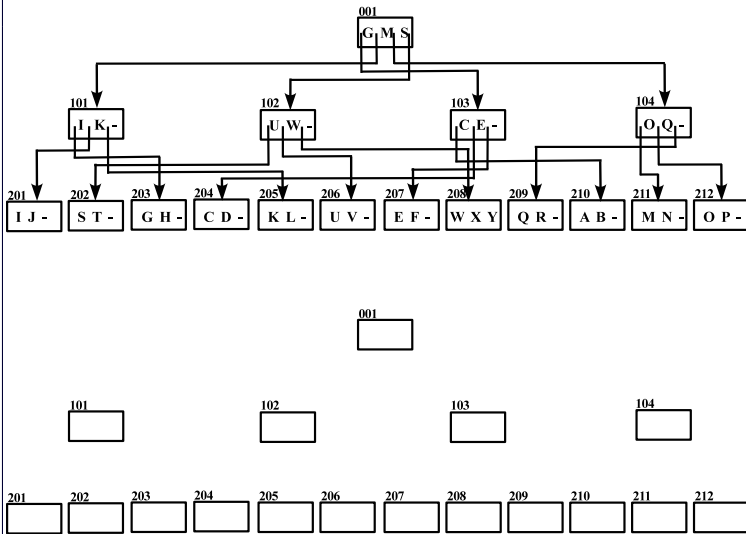
num_cover=1

num_cache=1

first search:

target= F

cover= I



Cached searches – Example (1)

l $Cache_l$

0 001 [$_{103}G_{101}M_{104}S_{102}$]

1 102 [$_{202}U_{206}W_{208}-$]

2 202 [ST-]

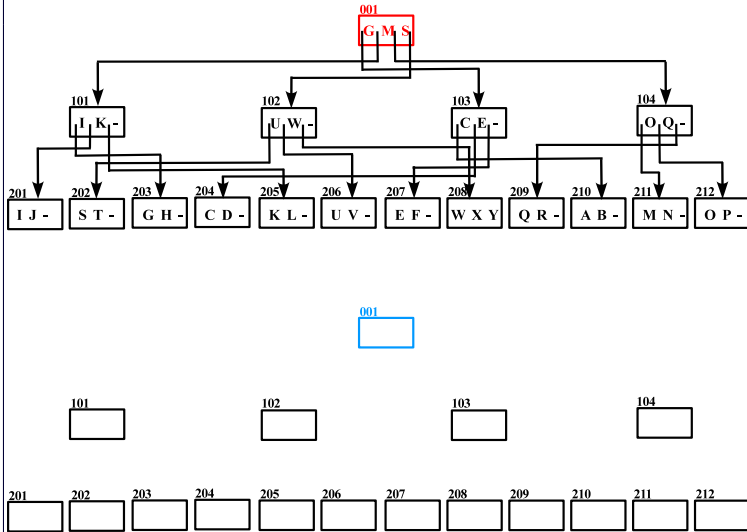
num_cover=1

num_cache=1

first search:

target= F

cover= I



Cached searches – Example (1)

l | $Cache_l$

0 | 001 [₁₀₃G₁₀₁M₁₀₄S₁₀₂]

1 | 102 [₂₀₂U₂₀₆W₂₀₈- -]

2 | 202 [ST-]

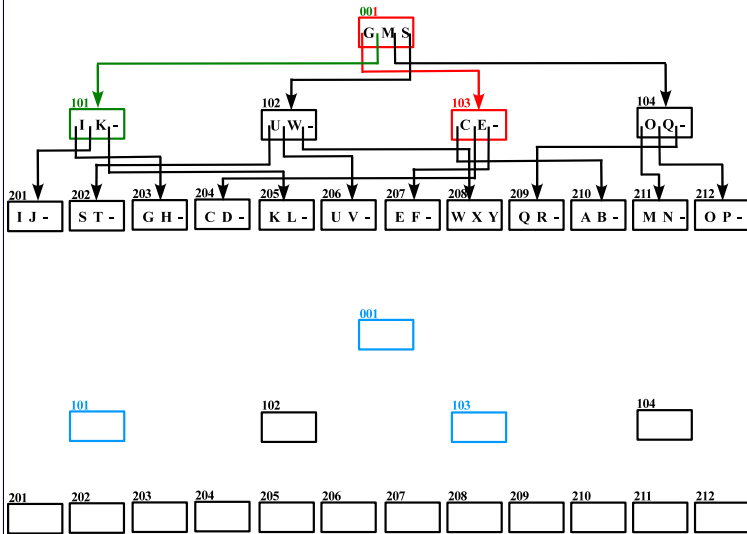
num_cover=1

num_cache=1

first search:

target= F

cover= I



Cached searches – Example (1)

l | $Cache_l$

0 | 001 [₁₀₃G₁₀₁M₁₀₄S₁₀₂]

1 | 103 [₂₁₀C₂₀₄E₂₀₇- -]

2 | 202 [ST-]

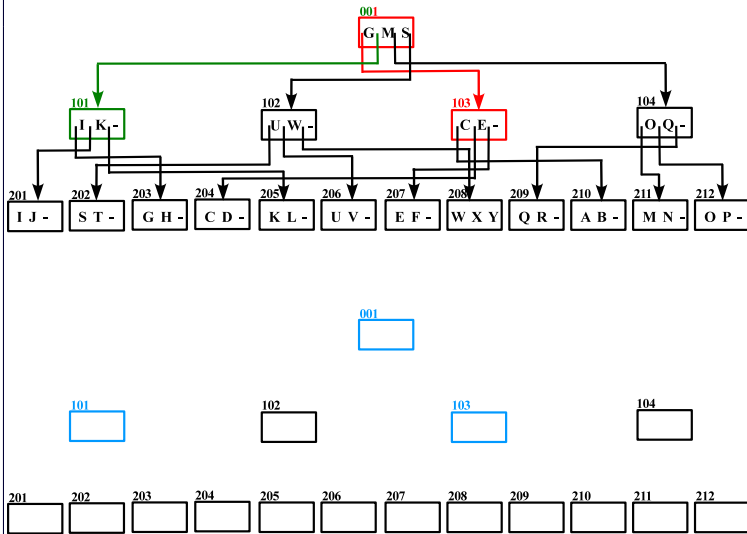
num_cover=1

num_cache=1

first search:

target= F

cover= I



Cached searches – Example (1)

l | $Cache_l$

0 | 001 [₁₀₃G₁₀₁M₁₀₄S₁₀₂]

1 | 103 [₂₁₀C₂₀₄E₂₀₇- -]

2 | 202 [ST-]

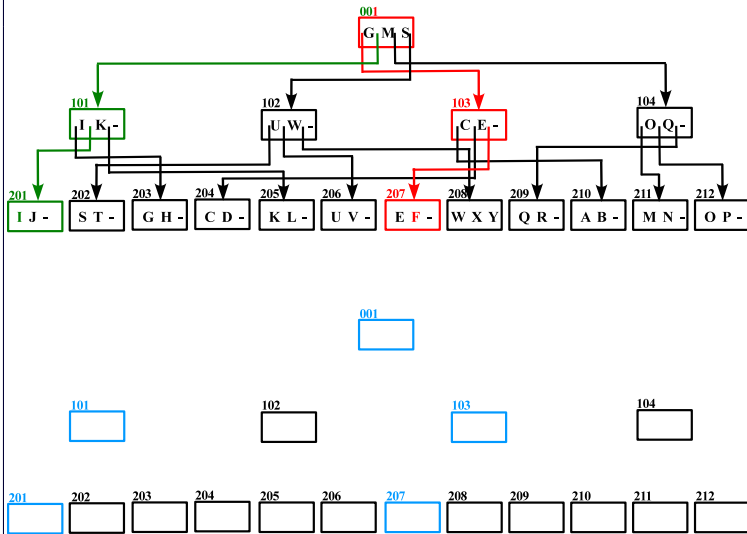
num_cover=1

num_cache=1

first search:

target= F

cover= I



Cached searches – Example (1)

l | $Cache_l$

0 | 001 [₁₀₃G₁₀₁M₁₀₄S₁₀₂]

1 | 103 [₂₁₀C₂₀₄E₂₀₇- -]

2 | 207 [EF-]

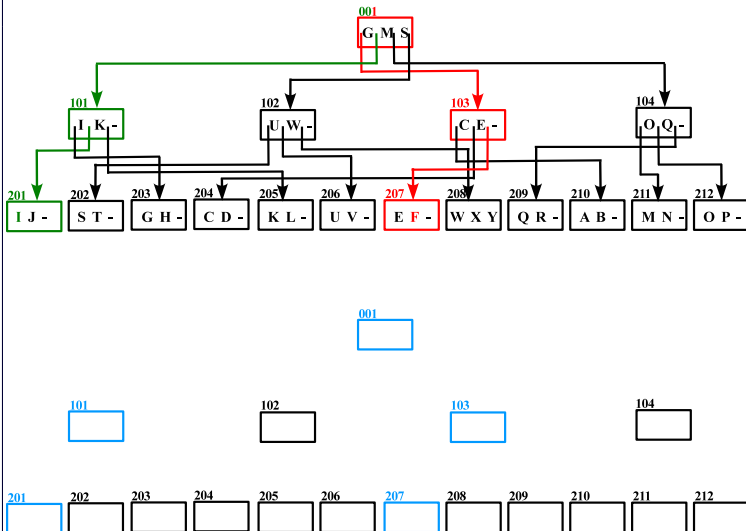
num_cover=1

num_cache=1

first search:

target= F

cover= I



Cached searches – Example (2)

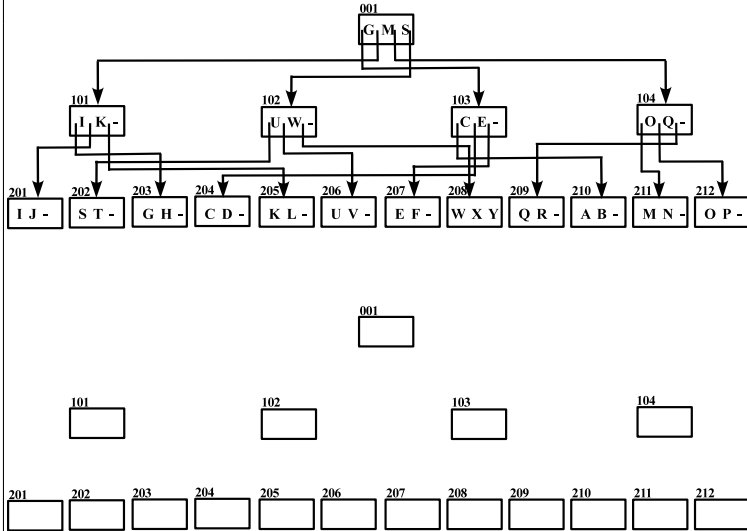
l	$Cache_l$
0	$001 [_{103}G_{101}M_{104}S_{102}]$
1	$103 [_{210}C_{204}E_{207} - -]$
2	$207 [EF-]$

num_cover=1
num_cache=1

second search:

target= F

covers= M,W



Cached searches – Example (2)

l | $Cache_l$

0 | 001 [₁₀₃G₁₀₁M₁₀₄S₁₀₂]

1 | 103 [₂₁₀C₂₀₄E₂₀₇- -]

2 | 207 [EF-]

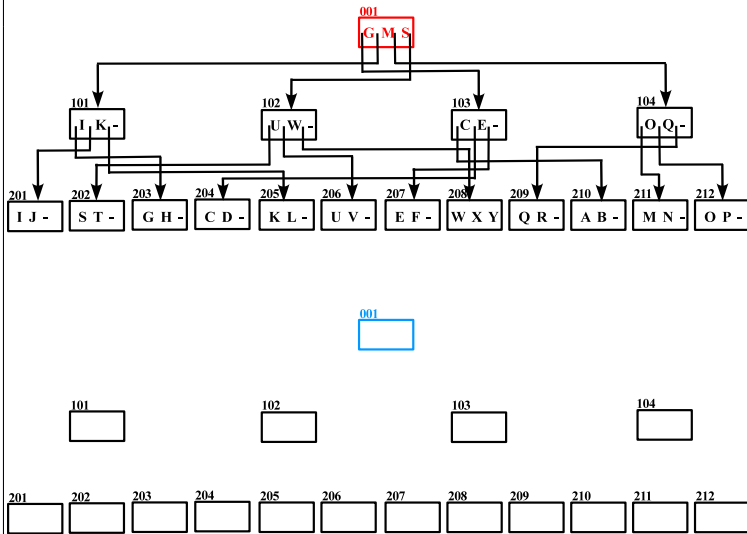
num_cover=1

num_cache=1

second search:

target= F

covers= M,W



Cached searches – Example (2)

l	$Cache_l$
0	001 [103 G101 M104 S102]
1	103 [210 C204 E207 - -]
2	207 [EF-]

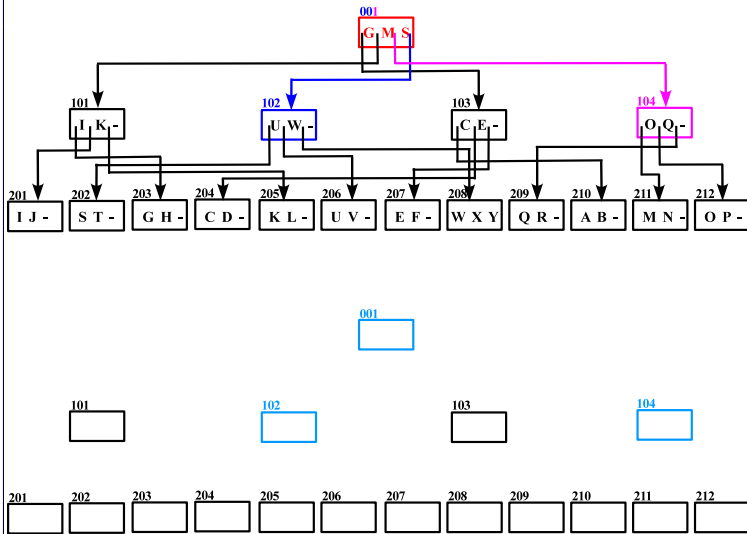
num_cover=1

num_cache=1

second search:

target= F

covers= M,W



Cached searches – Example (2)

l	$Cache_l$
0	001 $[_{103}G_{101}M_{104}S_{102}]$
1	103 $[_{210}C_{204}E_{207}- -]$
2	207 $[EF-]$

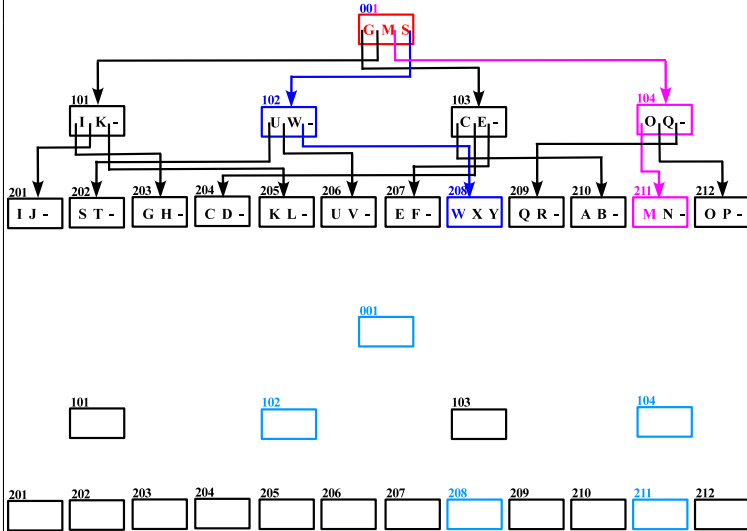
num_cover=1

num_cache=1

second search:

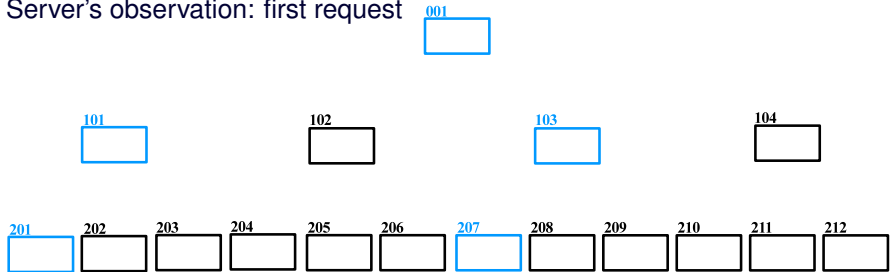
target= F

covers= M,W

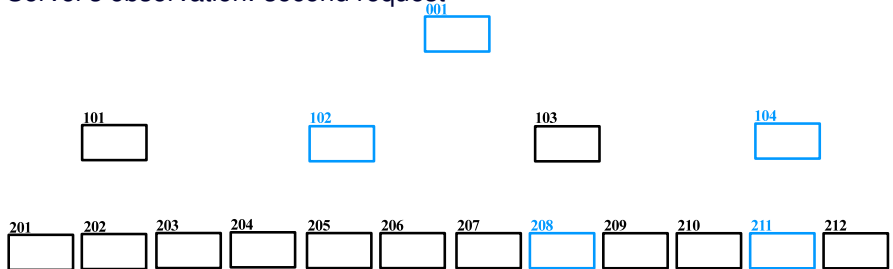


Cached searches – No intersection attack

Server's observation: first request



Server's observation: second request



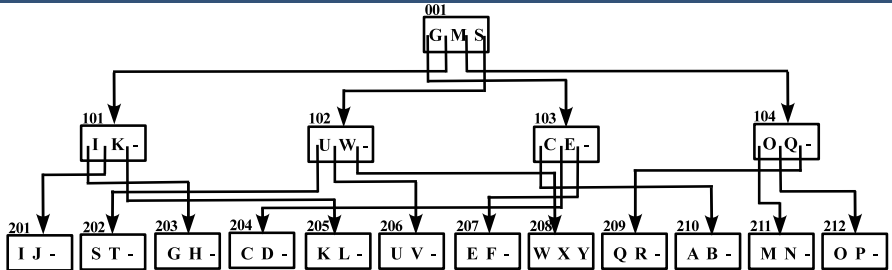
Cached searches – Protection offered

- + Caching helps in counteracting short term intersection attacks
 - o e.g., the observations of the server on the two previous requests would be $\{(001); (101,103); (201,207)\}$ and $\{(001); (102,104); (208,211)\}$
 - $\implies$ the server would not be able to determine whether the two requests aim at the same target
- Caching does not prevent intersection attacks on observations that go beyond the size of the cache
- A long history of observations will allow the server to reconstruct the topology (parent-child relationship) of the shuffle index

Shuffling

- Shuffling breaks the one-to-one correspondence blocks-nodes by exchanging the content among nodes (and therefore blocks)
 - Shuffling requires node decryption and re-encryption
 - encrypted text corresponding to a given node changes at each access (different node identifier and salt)
 - The contents of all blocks read in the execution of an access and the nodes in cache are exchanged
 - The shuffled blocks are rewritten back on the server
- ⇒ node shuffling at a given level requires to update the parents of the nodes

Shuffling – Example



001

101

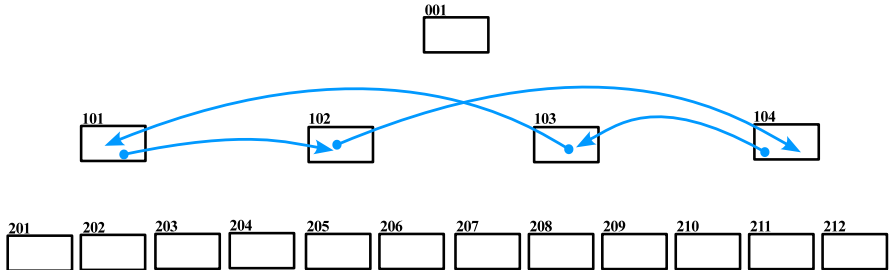
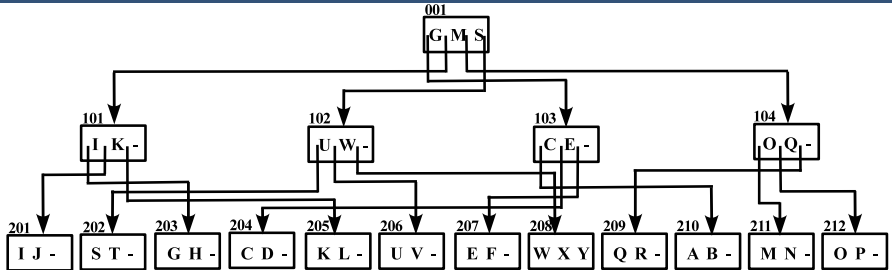
102

103

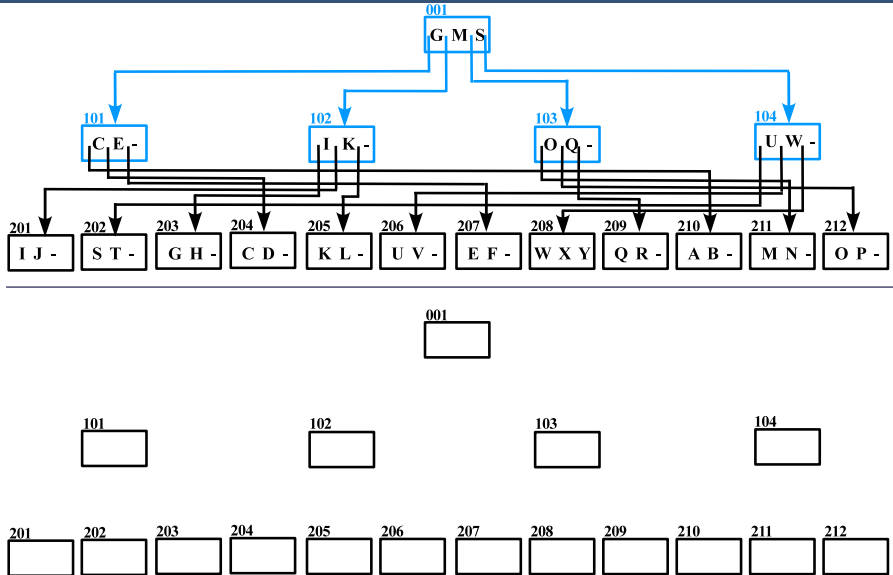
104

201 202 203 204 205 206 207 208 209 210 211 212

Shuffling – Example



Shuffling – Example



Access execution and shuffle index management

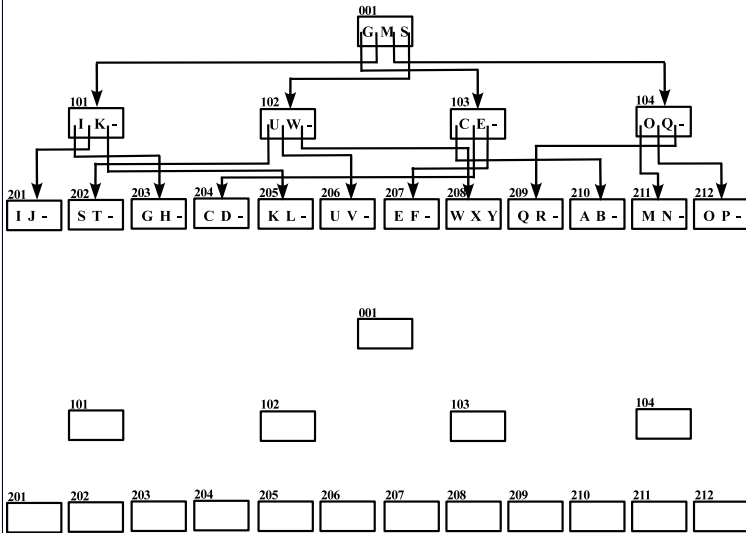
Let v be the target value. Determine $\text{num_cover}+1$ cover values and for each level l of the shuffle index:

- determine the identifiers (ToRead_ids) of the blocks in the path to v and cover values
- if the node in the path to v does not belong to Cache_l (cache miss), only num_cover cover searches are performed
- send to the server a request for the blocks with identifier in ToRead_ids and decrypt their content (set Read of nodes)
- shuffle nodes in Read and in Cache_l according to a permutation π
- update the pointers of the parents of the shuffled nodes
- update Cache_l by inserting the most recently accessed node in the path to v (only if a cache miss occurred)

Access execution – Example

l	$Cache_l$
0	001 $[_{103}G_{101}M_{104}S_{102}]$
1	101 $[_{203}I_{201}K_{205}- -]$ 103 $[_{210}C_{204}E_{207}- -]$
2	203 $[GH-]$ 210 $[AB-]$

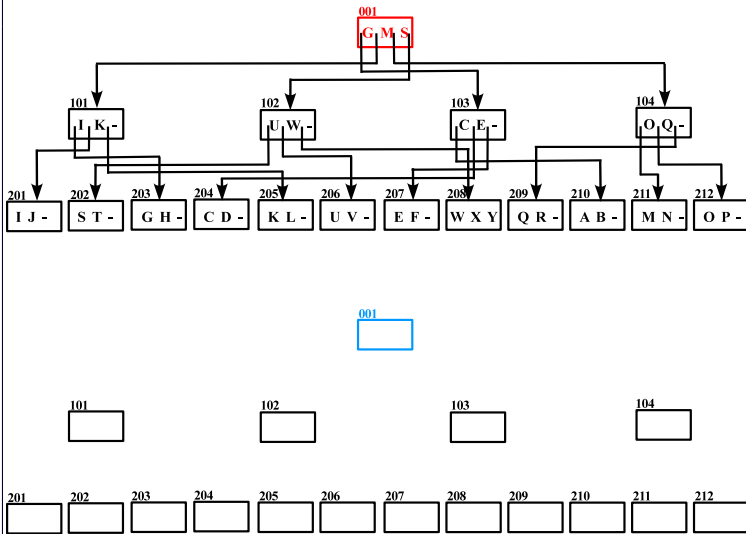
num_cover=1
 num_cache=2
 target= **F**
 covers= **S**,**M**



Access execution – Example

l	$Cache_l$
0	001 [₁₀₃ G ₁₀₁ M ₁₀₄ S ₁₀₂]
1	101 [₂₀₃ I ₂₀₁ K ₂₀₅ - -] 103 [₂₁₀ C ₂₀₄ E ₂₀₇ - -]
2	203 [GH-] 210 [AB-]

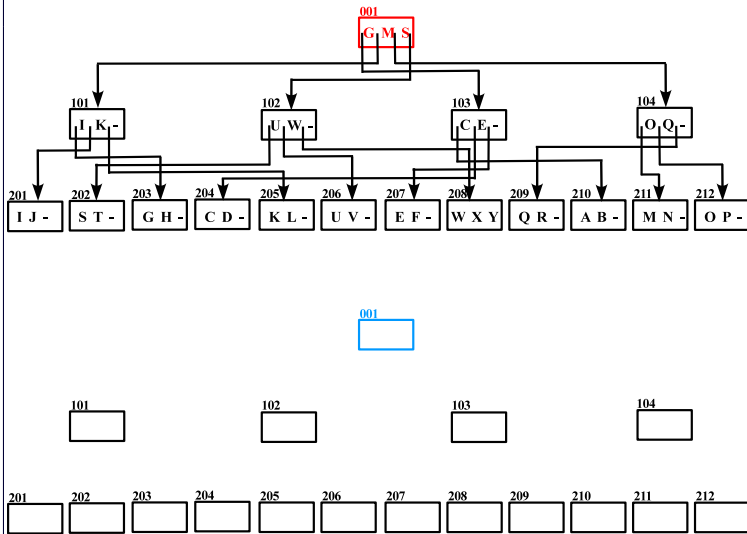
num_cover=1
 num_cache=2
 target= F
 covers= S,M



Access execution – Example

l	$Cache_l$
0	001 $[_{103}G_{101}M_{104}S_{102}]$
1	101 $[_{203}I_{201}K_{205}- -]$ 103 $[_{210}C_{204}E_{207}- -]$
2	203 $[GH-]$ 210 $[AB-]$

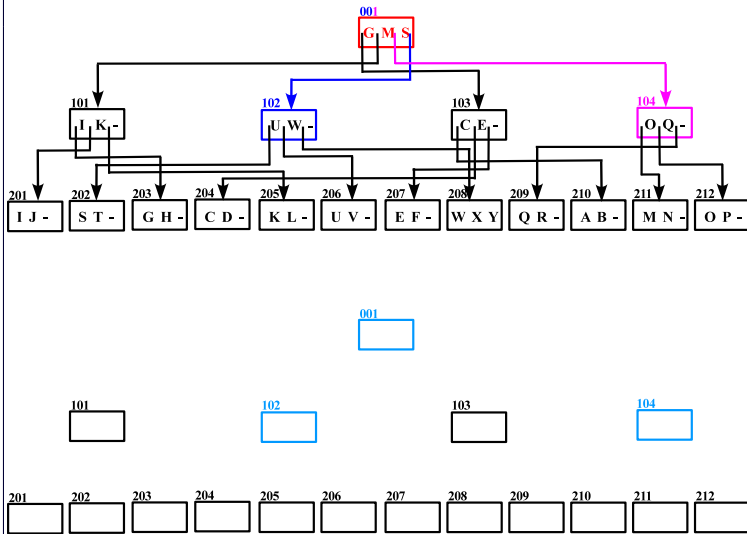
num_cover=1
 num_cache=2
 target= F
 covers= S,M



Access execution – Example

l	$Cache_l$
0	001 $[_{103}G_{101}M_{104}S_{102}]$
1	101 $[_{203}I_{201}K_{205}- -]$ 103 $[_{210}C_{204}E_{207}- -]$
2	203 $[GH-]$ 210 $[AB-]$

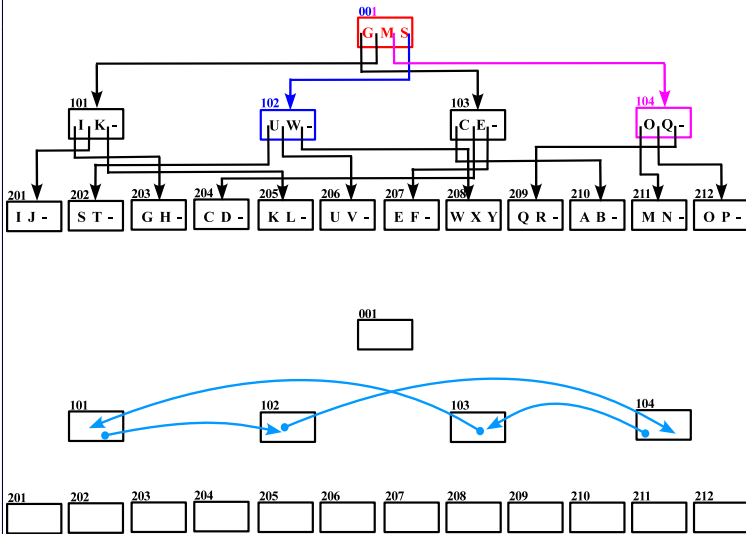
num_cover=1
 num_cache=2
 target= F
 covers= S,M



Access execution – Example

l	$Cache_l$
0	001 $[_{103}G_{101}M_{104}S_{102}]$
1	101 $[_{203}I_{201}K_{205}- -]$ 103 $[_{210}C_{204}E_{207}- -]$
2	203 $[GH-]$ 210 $[AB-]$

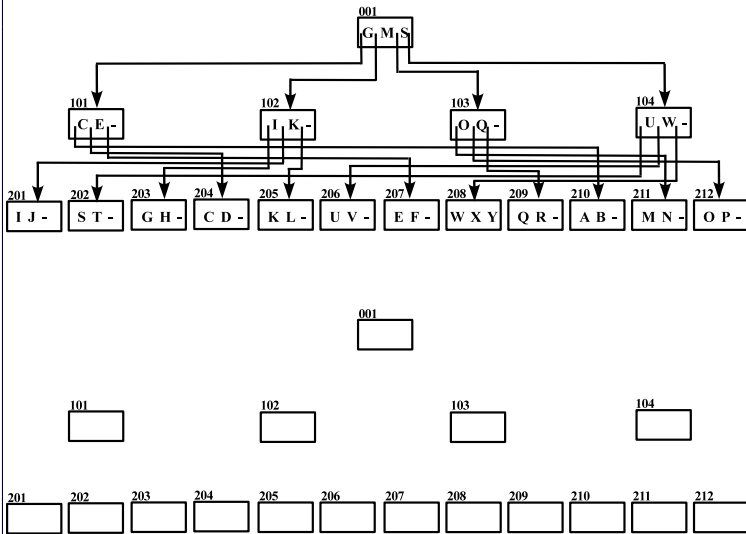
num_cover=1
 num_cache=2
 target= **F**
 covers= **S**, **M**



Access execution – Example

l	$Cache_l$
0	001 $[_{101}G_{102}M_{103}S_{104}]$
1	102 $[_{203}I_{201}K_{205}- -]$ 101 $[_{210}C_{204}E_{207}- -]$
2	203 $[GH-]$ 210 $[AB-]$

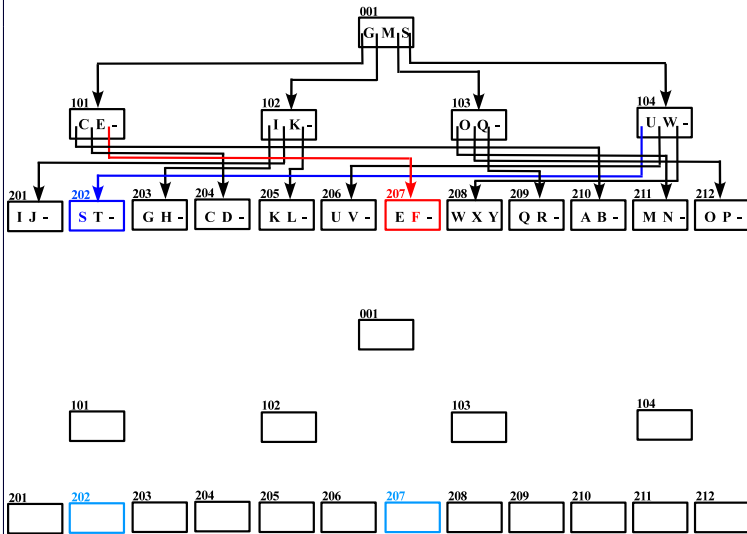
num_cover=1
 num_cache=2
 target= **F**
 covers= **S**,**M**



Access execution – Example

l	$Cache_l$
0	001 $[_{101}G_{102}M_{103}S_{104}]$
1	102 $[_{203}I_{201}K_{205}- -]$ 101 $[_{210}C_{204}E_{207}- -]$
2	203 $[GH-]$ 210 $[AB-]$

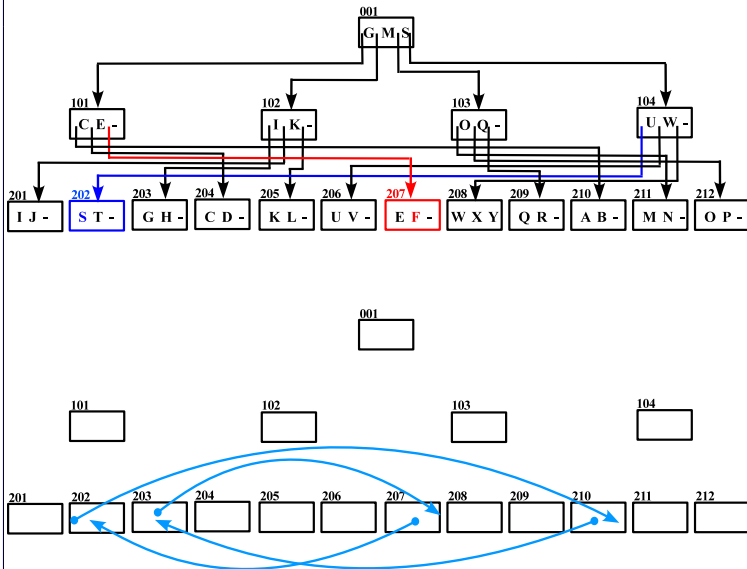
num_cover=1
 num_cache=2
 target= **F**
 covers= **S,M**



Access execution – Example

l	$Cache_l$
0	001 $[_{101}G_{102}M_{103}S_{104}]$
1	102 $[_{203}I_{201}K_{205}- -]$ 101 $[_{210}C_{204}E_{207}- -]$
2	203 $[GH-]$ 210 $[AB-]$

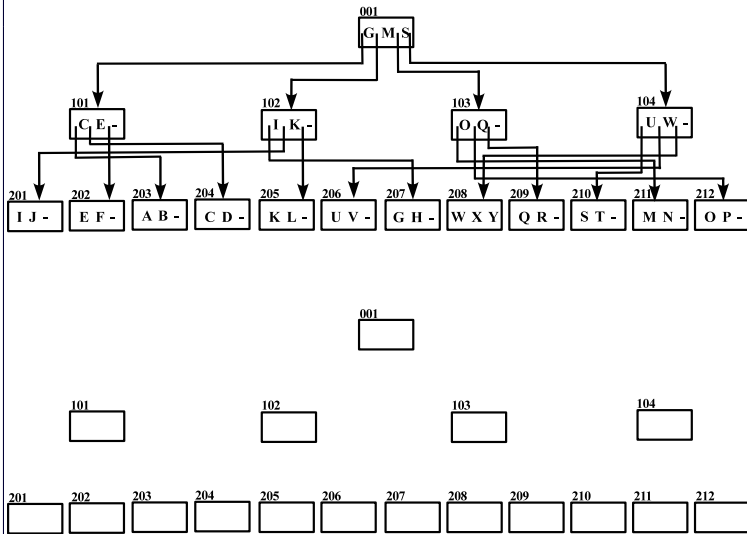
num_cover=1
 num_cache=2
 target= **F**
 covers= **S,M**



Access execution – Example

l	$Cache_l$
0	$001 \begin{bmatrix} 101 G_{102} M_{103} S_{104} \end{bmatrix}$
1	$102 \begin{bmatrix} 207 I_{201} K_{205} - - \end{bmatrix}$ $101 \begin{bmatrix} 203 C_{204} E_{202} - - \end{bmatrix}$
2	$207 \begin{bmatrix} GH- \end{bmatrix}$ $202 \begin{bmatrix} EF- \end{bmatrix}$

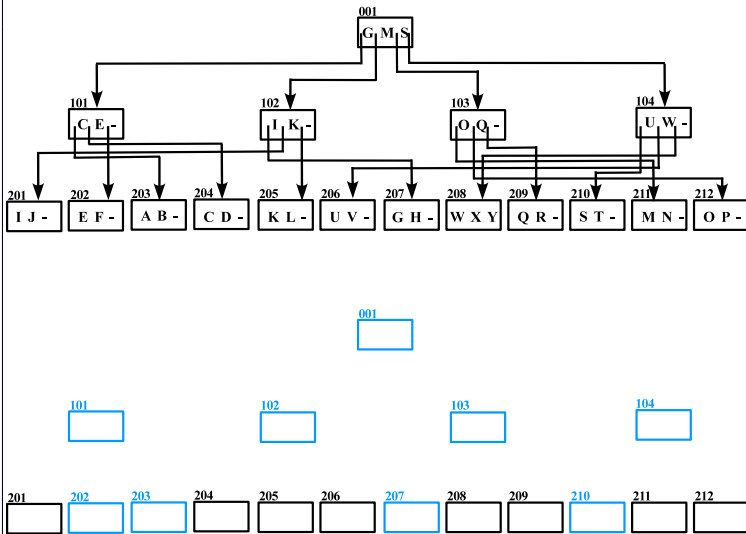
num_cover=1
 num_cache=2
 target= F
 covers= S,M



Access execution – Example

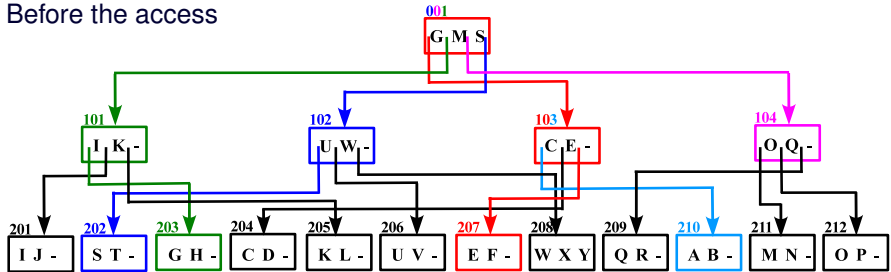
l	$Cache_l$
0	$001 \begin{bmatrix} 101 G_{102} M_{103} S_{104} \end{bmatrix}$
1	$102 \begin{bmatrix} 207 I_{201} K_{205} - - \end{bmatrix}$ $101 \begin{bmatrix} 203 C_{204} E_{202} - - \end{bmatrix}$
2	$207 \begin{bmatrix} GH- \end{bmatrix}$ $202 \begin{bmatrix} EF- \end{bmatrix}$

num_cover=1
num_cache=2
target= F
covers= S,M

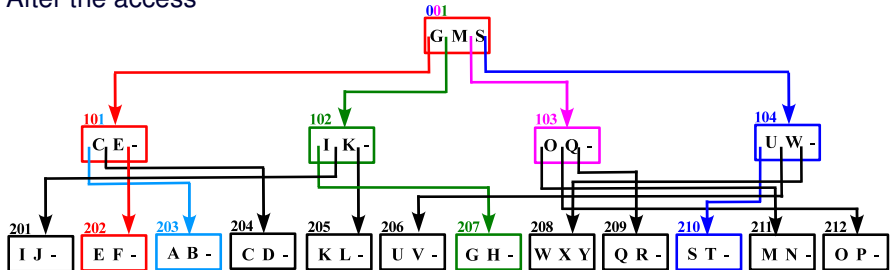


Access execution – Impact on the logical index

Before the access



After the access



Protection analysis

- **Degradation due to shuffling:** shuffling degrades any information the server may possess on the correspondence between nodes and blocks
- **Access confidentiality:** every time an access is performed any information on the specific access has to be divided among $\text{num_cover} + 1$ nodes and shuffling destroys the correspondence nodes-blocks
- **Pattern confidentiality:** accesses separated by a significant number of steps cannot be recognized (shuffling):
 - protection by covers and cache (short term)
 - protection by covers and shuffling (long term)

Protection vs performance

- Protection comes at a cost:
 - one read access implies $\text{num_cover} + \text{num_cache} + 1$ writes back to the server
 - + better performance with respect to Path ORAM
 - + no solution providing support for access and pattern confidentiality offers comparable performance (even in a WAN configuration)

Extensions to the shuffle index

The shuffle index can be extended to efficiently:

- support concurrent accesses ([delta versions](#)) [DFPPS-11b]
- operate on [multiple servers](#) for storing and accessing data ([shadows](#)) [DFPPS-13]

Integrity of Data Storage and Computation

S. De Capitani di Vimercati, S. Foresti, S. Jajodia, S. Paraboschi, R. Sassi, P. Samarati, "Sentinels and Twins: Effective Integrity Assessment for Distributed Computation," in *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 34, n. 1, January 2023, pp. 108-122.

S. De Capitani di Vimercati, S. Foresti, S. Jajodia, S. Paraboschi, P. Samarati, "Integrity for Join Queries in the Cloud," in *IEEE Transactions on Cloud Computing (TCC)*, vol. 1, n. 2, July-December 2013, pp. 187-200.

Integrity of storage and query computation – 1

- Data owner and users need mechanisms that provide integrity for query results:
 - **correctness**: computed on genuine data
 - **completeness**: computed on the whole data collection
 - **freshness**: computed on the most recent version of the data
- Two approaches:
 - **deterministic**: uses **authenticated data structures** (e.g., signature chains, Merkle hash trees, skip lists) or encryption-based solutions (e.g., verifiable homomorphic encryption schema [LDPW-14])
 - **probabilistic**: exploits insertion of **fake tuples** in query results, **replication of tuples** in query results, **pre-computed tokens** (e.g., [DFJPS-13b,DFJPS-14,DFJLPS-14b,XWYM-07])

Integrity of storage and query computation – 2

- Other approaches consider the verification of the integrity of query results of complex queries ([joins](#)):
 - [fake tuples](#) [XWYM-07]
 - spurious tuples
 - network overhead
 - [Merkle hash tree](#) or its variations [LHKR-06 YPPK-09]
 - support only joins on which the Merkle hash tree has been constructed

Merkle hash tree

- Binary tree where:
 - each leaf contains the hash of one tuple
 - each internal node contains the result of the hash of the concatenation of its children
- The hash function used to build the tree is collision-resistant
- The root is signed by the data owner and communicated to authorized users
- Tuples in the leaves are ordered according to the value of the attribute A on which the tree is defined
- The tree is created by the data owner and stored at the server

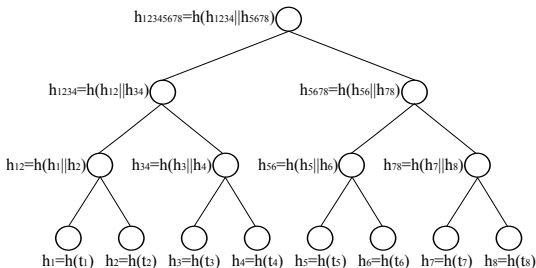
Merkle hash tree – Example

Patients

	<u>SSN</u>	Name	Disease
t_1	123-45-6789	Alice	Asthma
t_2	234-56-7891	Bob	Asthma
t_3	345-67-8912	Carol	Asthma
t_4	456-78-9123	David	Bronchitis
t_5	567-89-1234	Eva	Bronchitis
t_6	678-91-2345	Frank	Gastritis
t_7	789-12-3456	Gary	Gastritis
t_8	891-23-4567	Hilary	Diabetes

Merkle hash tree – Example

Patients			
	SSN	Name	Disease
t_1	123-45-6789	Alice	Asthma
t_2	234-56-7891	Bob	Asthma
t_3	345-67-8912	Carol	Asthma
t_4	456-78-9123	David	Bronchitis
t_5	567-89-1234	Eva	Bronchitis
t_6	678-91-2345	Frank	Gastritis
t_7	789-12-3456	Gary	Gastritis
t_8	891-23-4567	Hilary	Diabetes



Merkle hash tree over attribute **SSN**

Merkle hash tree verification

- The Merkle hash tree defined over A supports the verification of **equality** and **range** queries over A
- The server returns, together with the query result, a **verification object** (hash of other tuples allowing to derive the hash of the root)
- The client uses the verification object and query result to **recompute the root** of the tree
- The query result is **correct and complete** iff the computed root is **the same** as the one she knows
 - if a tuple is **not correct** or is **missing** from the query result, the recomputed root value is **not** the same as the one known to the client

Merkle hash tree verification – Example

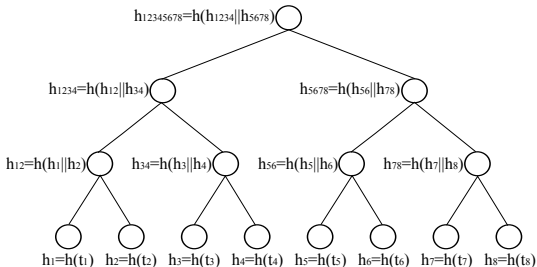
SELECT *

FROM Patients

WHERE SSN = '345-67-8912'

Patients

	<u>SSN</u>	Name	Disease
t_1	123-45-6789	Alice	Asthma
t_2	234-56-7891	Bob	Asthma
t_3	345-67-8912	Carol	Asthma
t_4	456-78-9123	David	Bronchitis
t_5	567-89-1234	Eva	Bronchitis
t_6	678-91-2345	Frank	Gastritis
t_7	789-12-3456	Gary	Gastritis
t_8	891-23-4567	Hilary	Diabetes



Merkle hash tree verification – Example

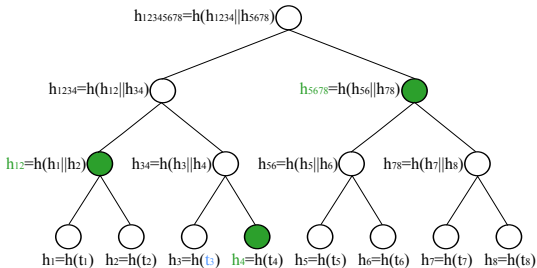
SELECT *

FROM Patients

WHERE SSN = '345-67-8912'

Patients

	SSN	Name	Disease
t_1	123-45-6789	Alice	Asthma
t_2	234-56-7891	Bob	Asthma
t_3	345-67-8912	Carol	Asthma
t_4	456-78-9123	David	Bronchitis
t_5	567-89-1234	Eva	Bronchitis
t_6	678-91-2345	Frank	Gastritis
t_7	789-12-3456	Gary	Gastritis
t_8	891-23-4567	Hilary	Diabetes



Result: t_3

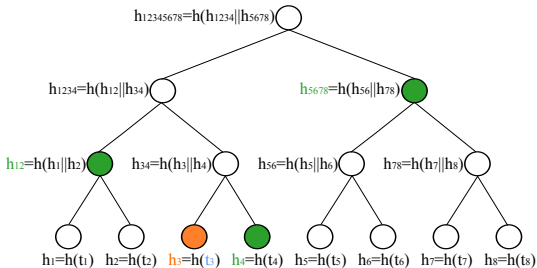
Verification Object: h_4 , h_{12} , h_{5678}

Merkle hash tree verification – Example

SELECT *
FROM Patients
WHERE SSN = '345-67-8912'

Patients

	SSN	Name	Disease
t_1	123-45-6789	Alice	Asthma
t_2	234-56-7891	Bob	Asthma
t_3	345-67-8912	Carol	Asthma
t_4	456-78-9123	David	Bronchitis
t_5	567-89-1234	Eva	Bronchitis
t_6	678-91-2345	Frank	Gastritis
t_7	789-12-3456	Gary	Gastritis
t_8	891-23-4567	Hilary	Diabetes



Result: t_3

Verification Object: h_4 , h_{12} , h_{5678}

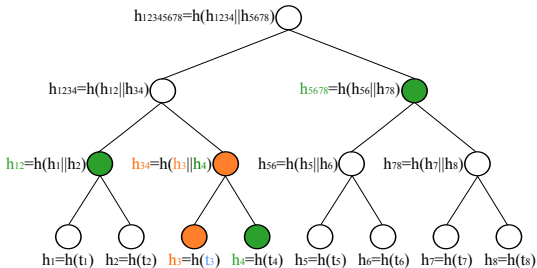
$h_3 = h(t_3)$

Merkle hash tree verification – Example

SELECT *
FROM Patients
WHERE SSN = '345-67-8912'

Patients

	SSN	Name	Disease
t_1	123-45-6789	Alice	Asthma
t_2	234-56-7891	Bob	Asthma
t_3	345-67-8912	Carol	Asthma
t_4	456-78-9123	David	Bronchitis
t_5	567-89-1234	Eva	Bronchitis
t_6	678-91-2345	Frank	Gastritis
t_7	789-12-3456	Gary	Gastritis
t_8	891-23-4567	Hilary	Diabetes



Result: t_3

Verification Object: h_4 , h_{12} , h_{5678}

$h_3 = h(t_3)$

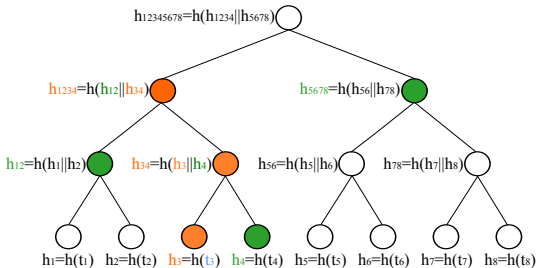
$h_{34} = h(h_3 || h_4)$

Merkle hash tree verification – Example

SELECT *
FROM Patients
WHERE SSN = '345-67-8912'

Patients

	SSN	Name	Disease
t_1	123-45-6789	Alice	Asthma
t_2	234-56-7891	Bob	Asthma
t_3	345-67-8912	Carol	Asthma
t_4	456-78-9123	David	Bronchitis
t_5	567-89-1234	Eva	Bronchitis
t_6	678-91-2345	Frank	Gastritis
t_7	789-12-3456	Gary	Gastritis
t_8	891-23-4567	Hilary	Diabetes



Result: t_3

Verification Object: h_4 , h_{12} , h_{5678}

$$h_3 = h(t_3)$$

$$h_{34} = h(h_3 || h_4)$$

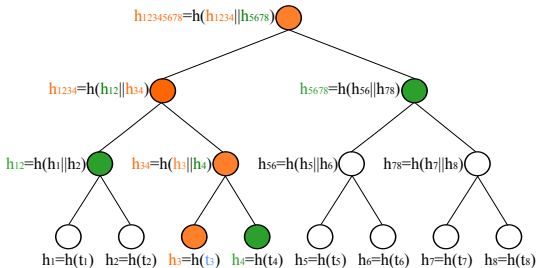
$$h_{1234} = h(h_{12} || h_{34})$$

Merkle hash tree verification – Example

SELECT *
FROM Patients
WHERE SSN = '345-67-8912'

Patients

	SSN	Name	Disease
t_1	123-45-6789	Alice	Asthma
t_2	234-56-7891	Bob	Asthma
t_3	345-67-8912	Carol	Asthma
t_4	456-78-9123	David	Bronchitis
t_5	567-89-1234	Eva	Bronchitis
t_6	678-91-2345	Frank	Gastritis
t_7	789-12-3456	Gary	Gastritis
t_8	891-23-4567	Hilary	Diabetes



Result: t_3

Verification Object: h_4 , h_{12} , h_{5678}

$$h_3 = h(t_3)$$

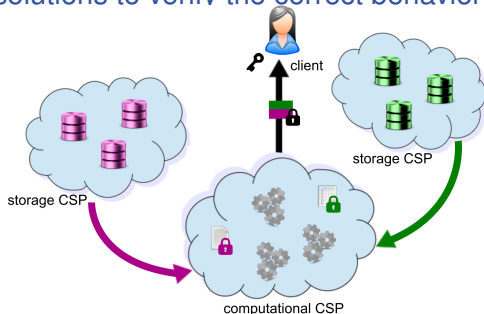
$$h_{34} = h(h_3 || h_4)$$

$$h_{1234} = h(h_{12} || h_{34})$$

$$h_{12345678} = h(h_{1234} || h_{5678})$$

Computation with multiple providers

- Different CSPs are available on the market, offering a **variety of services** (e.g., storage, computation) at different prices
- Users can select the CSP that better matches their security, economic, and functional requirements
- Multiple CSPs can help enhancing security **but**
⇒ **need solutions to verify the correct behavior of these CSPs**



Probabilistic approach for join queries

- A client, with the cooperation of the storage servers, can assess the integrity of joins performed by a computational cloud
- Protection techniques [DFJPS-13b,DFJPS-14]:
 - **encryption** makes data unintelligible
 - **markers**, fake tuples not recognizable as such by the computational cloud (and not colliding with real tuples)
 - **twins**, replication of existing tuples
 - **salts/buckets**, replications with salts (at side 1) and dummy tuples (at side many) to flatten occurrences of matches in 1:n joins
- A marker missing or a twin appearing solo $\implies$ **integrity violation**
- **Probabilistic guarantee** depending on the amount of control (markers and twins) inserted

Probabilistic approach for join queries – Example

CLIENT

COMPUTATIONAL CLOUD

L

STORAGE SERVER S_l

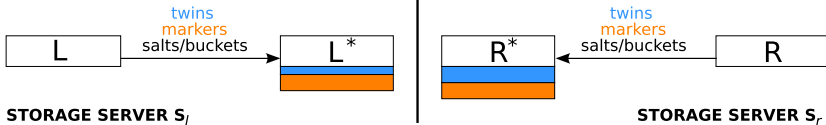
R

STORAGE SERVER S_r

Probabilistic approach for join queries – Example

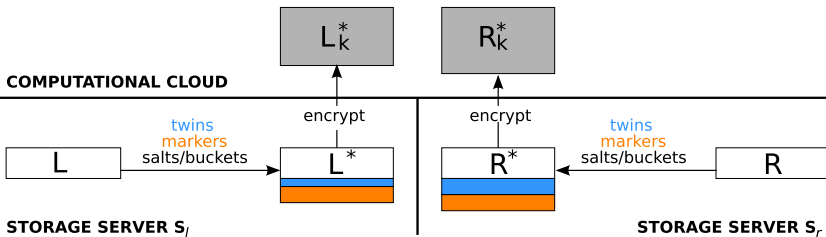
CLIENT

COMPUTATIONAL CLOUD



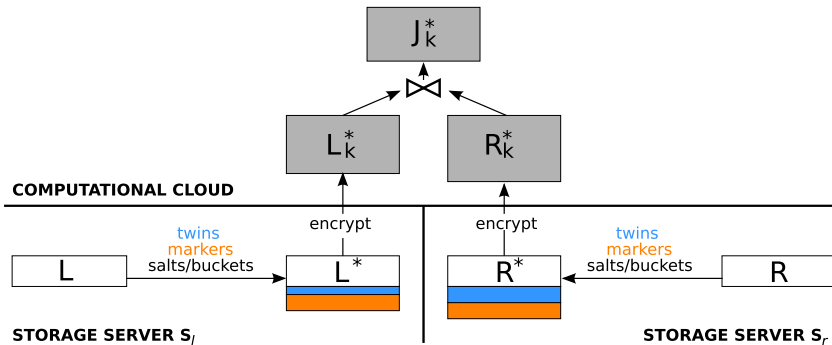
Probabilistic approach for join queries – Example

CLIENT

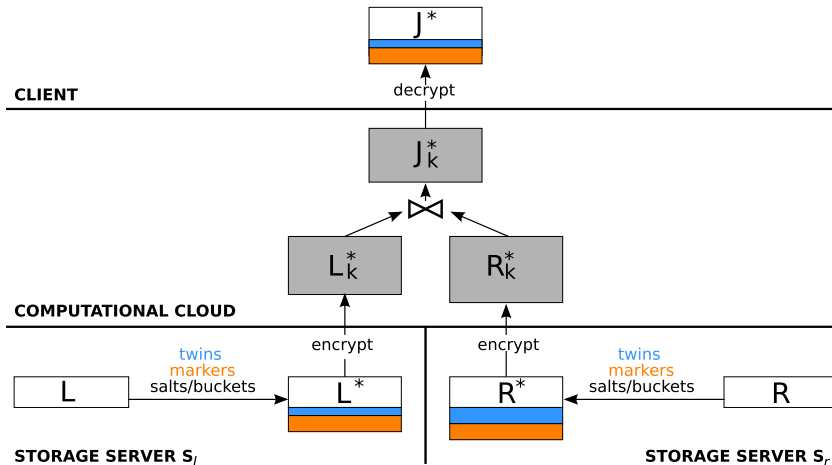


Probabilistic approach for join queries – Example

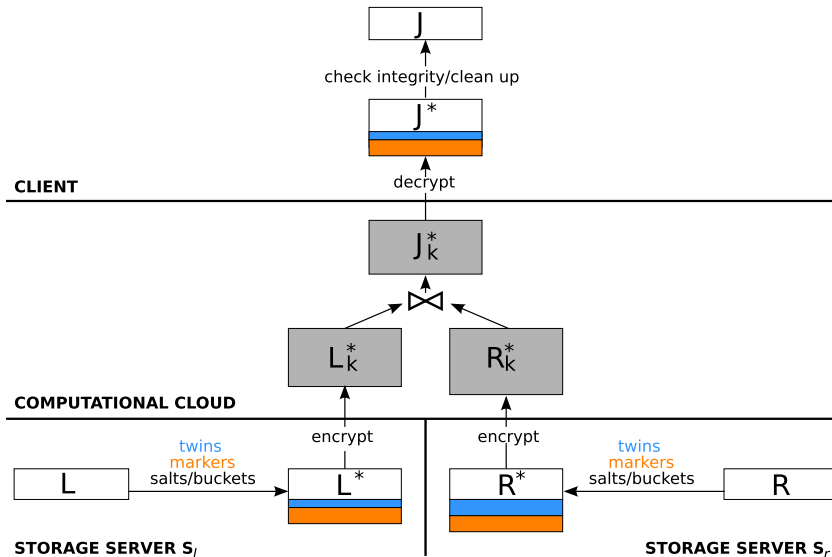
CLIENT



Probabilistic approach for join queries – Example



Probabilistic approach for join queries – Example



On-the-fly encryption

- Server S **encrypts** $B(I, Att)$, obtaining $B_k(I_k, B.Tuple_k)$
 - For each t in B , there is τ in B_k : $\tau[I_k]=E_k(t[I])$ and $\tau[B.Tuple_k]=E_k(t)$
 - E is a **symmetric** encryption function with key k
 - k is defined by the client and **changes** at every query
- Encryption provides data **confidentiality**

R_l	
I	Attr
l_1 a	Ann
l_2 b	Beth
l_3 c	Cloe

R_r	
I	Attr
r_1 a	flu
r_2 a	asthma
r_3 b	ulcer
r_4 e	hernia
r_5 e	flu
r_6 e	cancer

J			
L.I	L.Attr	R.I	R.Attr
l_1 a	Ann	a	flu
l_1 a	Ann	a	asthma
l_2 b	Beth	b	ulcer

On-the-fly encryption

- Server S **encrypts** $B(I, Att)$, obtaining $B_k(I_k, B.Tuple_k)$
 - For each t in B , there is τ in B_k : $\tau[I_k]=E_k(t[I])$ and $\tau[B.Tuple_k]=E_k(t)$
 - E is a **symmetric** encryption function with key k
 - k is defined by the client and **changes** at every query
- Encryption provides data **confidentiality**

$$R_{lk}$$

I_k	$L.Tuple_k$
α	λ_1
β	λ_2
γ	λ_3

$$R_{rk}$$

I_k	$R.Tuple_k$
α	ρ_1
α	ρ_2
β	ρ_3
ε	ρ_4
ε	ρ_5
ε	ρ_6

$$J_k$$

$L.I_k$	$L.Attr_k$	$R.I_k$	$R.Attr_k$
α	λ_1	α	ρ_1
α	λ_1	α	ρ_2
β	λ_2	β	ρ_3

Markers

- Artificial tuples injected into R_l by S_l and R_r by S_r
 - not recognizable by the computational server
 - do not generate spurious tuples
 - inserted in a concerted manner to guarantee that they belong to the join result
- The absence of markers signals incompleteness of the join result

R_l	
I	Attr
l_1	a Ann
l_2	b Beth
l_3	c Cloe

R_r	
I	Attr
r_1	a flu
r_2	a asthma
r_3	b ulcer
r_4	e hernia
r_5	e flu
r_6	e cancer

J					
	L.I	L.Attr	R.I	R.Attr	
l_1	a	Ann	a	flu	r_1
l_1	a	Ann	a	asthma	r_2
l_2	b	Beth	b	ulcer	r_3

Markers

- Artificial tuples injected into R_l by S_l and R_r by S_r
 - not recognizable by the computational server
 - do not generate spurious tuples
 - inserted in a concerted manner to guarantee that they belong to the join result
- The absence of markers signals incompleteness of the join result

R_l^*

	I	Attr
l_1	a	Ann
l_2	b	Beth
l_3	c	Cloe
m_1	x	marker ₁

R_r^*

	I	Attr
r_1	a	flu
r_2	a	asthma
r_3	b	ulcer
r_4	e	hernia
r_5	e	flu
r_6	e	cancer
m_2	x	marker ₂

J^*

	L.I	L.Attr	R.I	R.Attr	
l_1	a	Ann	a	flu	r_1
l_1	a	Ann	a	asthma	r_2
l_2	b	Beth	b	ulcer	r_3
m_1	x	marker ₁	x	marker ₂	m_2

Twins

- Duplicates of tuples that satisfy condition C_{twin} that
 - is defined on the join attribute I
 - tunes the percentage p_t of twins
 - is defined by the client and communicated to S_l and S_r
- Twin pairs are not recognizable by the computational server
- A twin appearing solo signals incompleteness of the join result

R_l	
I	Attr
l_1 a	Ann
l_2 b	Beth
l_3 c	Cloe

R_r	
I	Attr
r_1 a	flu
r_2 a	asthma
r_3 b	ulcer
r_4 e	hernia
r_5 e	flu
r_6 e	cancer

J				
	L.I	L.Attr	R.I	R.Attr
l_1	a	Ann	a	flu
l_1	a	Ann	a	asthma
l_2	b	Beth	b	ulcer

Twins

- **Duplicates** of tuples that satisfy condition C_{twin} that
 - is defined on the **join attribute** I
 - tunes the **percentage** p_t of twins
 - is defined by the client and communicated to S_l and S_r
- Twin pairs are **not recognizable** by the computational server
- A twin appearing **solo** signals **incompleteness** of the join result

R_l^*

	I	Attr
l_1	a	Ann
l_2	b	Beth
l_3	c	Cloe
$\bar{l}_2$	$\bar{b}$	Beth

R_r^*

	I	Attr
r_1	a	flu
r_2	a	asthma
r_3	b	ulcer
r_4	e	hernia
r_5	e	flu
r_6	e	cancer
$\bar{r}_3$	$\bar{b}$	ulcer

J^*

	L.I	L.Attr	R.I	R.Attr	
l_1	a	Ann	a	flu	r_1
l_1	a	Ann	a	asthma	r_2
l_2	b	Beth	b	ulcer	r_3
$\bar{l}_2$	$\bar{b}$	Beth	$\bar{b}$	ulcer	$\bar{r}_3$

Salts and buckets

- Destroy recognizable frequencies of combinations in one-to-many joins
- Operate on original tuples, markers, and twins and can be adopted in alternative or in combination
- Salts
 - map different occurrences of the same join value on the side “many” of the join to a different encrypted value using a different salt
 - replicate each tuple on the side “one” of the join and combine replicas with different salts to guarantee the matching
- Buckets
 - insert dummy tuples on the side “many” of the join to guarantee flat frequency distribution of join attribute values

Salts and buckets – Example

- number of salts: 2
- maximum number of occurrences: 3

⇒ buckets with 2 tuples each

R_l^*	
I	Attr
l_1 a	Ann
l_2 b	Beth
l_3 c	Cloe

R_r^*	
I	Attr
r_1 a	flu
r_2 a	asthma
r_3 b	ulcer
r_4 e	hernia
r_5 e	flu
r_6 e	cancer

J^*					
	L.I	L.Attr	R.I	R.Attr	
l_1	a	Ann	a	flu	r_1
l_1	a	Ann	a	asthma	r_2
l_2	b	Beth	b	ulcer	r_3

Salts and buckets – Example

- number of salts: 2
- maximum number of occurrences: 3

⇒ buckets with 2 tuples each

$$R_l^*$$

	I	Attr
l_1	a	Ann
l_1'	a'	Ann'
l_2	b	Beth
l_2'	b'	Beth'
l_3	c	Cloe
l_3'	c'	Cloe'

$$R_r^*$$

	I	Attr
r_1	a	flu
r_2	a	asthma
r_3	b	ulcer
d_1	b	<i>dummy</i> ₁
r_4	e	hernia
r_5	e	flu
r_6	e'	cancer
d_2	e'	<i>dummy</i> ₂

$$J^*$$

	L.I	L.Attr	R.I	R.Attr	
l_1	a	Ann	a	flu	r_1
l_1	a	Ann	a	asthma	r_2
l_2	b	Beth	b	ulcer	r_3
l_2	b	Beth	b	<i>dummy</i> ₁	d_1

Query evaluation

The client shares with each server S_i a symmetric key k_i

- The client send the computational cloud a request to execute a join between the relations produced by S_l and S_r
- The relations to be produced by S_l and S_r are represented as two strings, encrypted with keys k_l and k_r , respectively, and to be forwarded by the computational cloud to the respective storage server, containing:
 - subquery to be executed by the storage server
 - query key k (on-the-fly encryption) to be used by the storage server to encrypt the relation sent to the computational cloud
 - number m of markers
 - percentage p_t of twins
 - number s of salts

Join execution – Example

	R_l	
	I	Attr
l_1	a	Ann
l_2	b	Beth
l_3	c	Cloe

	R_r	
	I	Attr
r_1	a	flu
r_2	a	asthma
r_3	b	ulcer
r_4	e	hernia
r_5	e	flu
r_6	e	cancer

Storage servers

Join execution – Example

	R_l^*	
	I	Attr
l_1	a	Ann
l_2	b	Beth
l_3	c	Cloe
$\bar{l}_2$	$\bar{b}$	Beth

	R_r^*	
	I	Attr
r_1	a	flu
r_2	a	asthma
r_3	b	ulcer
r_4	e	hernia
r_5	e	flu
r_6	e	cancer
$\bar{r}_3$	$\bar{b}$	ulcer

Storage servers

Join execution – Example

	R_l^*	
	I	Attr
l_1	a	Ann
l_2	b	Beth
l_3	c	Cloe
$\bar{l}_2$	$\bar{b}$	Beth
m_1	x	marker ₁

	R_r^*	
	I	Attr
r_1	a	flu
r_2	a	asthma
r_3	b	ulcer
r_4	e	hernia
r_5	e	flu
r_6	e	cancer
$\bar{r}_3$	$\bar{b}$	ulcer
m_2	x	marker ₂

Storage servers

Join execution – Example

	I	Attr
l_1	a	Ann
l_1'	a'	Ann'
l_2	b	Beth
l_2'	b'	Beth'
l_3	c	Cloe
l_3'	c'	Cloe'
$\bar{l}_2$	$\bar{b}$	Beth
$\bar{l}_2'$	$\bar{b}'$	Beth'
m_1	x	marker ₁

	I	Attr
r_1	a	flu
r_2	a	asthma
r_3	b	ulcer
d_1	b	dummy ₁
r_4	e	hernia
r_5	e	flu
r_6	e'	cancer
d_2	e'	dummy ₂
$\bar{r}_3$	$\bar{b}$	ulcer
$\bar{d}_1$	$\bar{b}$	dummy ₁
m_2	x	marker ₂
d_3	x	dummy ₃

Storage servers

Join execution – Example

	R_l^*
I	Attr
l_1	a Ann
l_1'	a' Ann'
l_2	b Beth
l_2'	b' Beth'
l_3	c Cloe
l_3'	c' Cloe'
$\bar{l}_2$	$\bar{b}$ Beth
$\bar{l}_2'$	$\bar{b}'$ Beth'
m_1	x marker ₁

	R_r^*
I	Attr
r_1	a flu
r_2	a asthma
r_3	b ulcer
d_1	b dummy ₁
r_4	e hernia
r_5	e flu
r_6	e' cancer
d_2	e' dummy ₂
$\bar{r}_3$	$\bar{b}$ ulcer
$\bar{d}_1$	$\bar{b}$ dummy ₁
m_2	x marker ₂
d_3	x dummy ₃

Storage servers

	R_{lk}^*
I_k	L.Tuple _k
α	λ_1
α'	λ_1'
β	λ_2
β'	λ_2'
γ	λ_3
γ'	λ_3'
$\bar{\beta}$	λ_2
$\bar{\beta}'$	λ_2'
χ	μ_1

	R_{rk}^*
I_k	R.Tuple _k
α	ρ_1
α	ρ_2
β	ρ_3
β	δ_1
ε	ρ_4
ε	ρ_5
ε'	ρ_6
ε'	δ_2
$\bar{\beta}$	$\bar{\rho}_3$
$\bar{\beta}$	δ_1
χ	μ_2
χ	δ_3

Computational cloud

Join execution – Example

	R_l^*
l	Attr
l_1	a Ann
l_1'	a' Ann'
l_2	b Beth
l_2'	b' Beth'
l_3	c Cloe
l_3'	c' Cloe'
$\bar{l}_2$	$\bar{b}$ Beth
$\bar{l}_2'$	$\bar{b}'$ Beth'
m_1	x marker ₁

	R_r^*
r	Attr
r_1	a flu
r_2	a asthma
r_3	b ulcer
d_1	b dummy ₁
r_4	e hernia
r_5	e flu
r_6	e' cancer
d_2	e' dummy ₂
$\bar{r}_3$	$\bar{b}$ ulcer
$\bar{d}_1$	$\bar{b}$ dummy ₁
m_2	x marker ₂
d_3	x dummy ₃

Storage servers

	R_l^*
l_k	L.Tuple _k
α	λ_1
α'	λ_1'
β	λ_2
β'	λ_2'
γ	λ_3
γ'	λ_3'
$\bar{\beta}$	λ_2
$\bar{\beta}'$	λ_2'
χ	μ_1

	R_r^*
l_k	R.Tuple _k
α	ρ_1
α	ρ_2
β	ρ_3
β	δ_1
ε	ρ_4
ε	ρ_5
ε'	ρ_6
ε'	δ_2
$\bar{\varepsilon}$	$\bar{\rho}_3$
$\bar{\beta}$	δ_1
χ	μ_2
χ	δ_3

J_k^*

L.I. _k	L.Tuple _k	R.I. _k	R.Tuple _k
α	λ_1	α	ρ_1
α	λ_1	α	ρ_2
β	λ_2	β	ρ_3
β	λ_2	β	δ_1
$\bar{\beta}$	λ_2	$\bar{\beta}$	$\bar{\rho}_3$
$\bar{\beta}$	λ_2	$\bar{\beta}$	δ_1
χ	μ_1	χ	μ_2
χ	μ_1	χ	δ_3

Computational cloud

Join execution – Example

$$R_l^*$$

I	Attr
l_1	a Ann
l_1'	a' Ann'
l_2	b Beth
l_2'	b' Beth'
l_3	c Cloe
l_3'	c' Cloe'
$\bar{l}_2$	$\bar{b}$ Beth
$\bar{l}_2'$	$\bar{b}'$ Beth'
m_1	x marker ₁

$$R_r^*$$

I	Attr
r_1	a flu
r_2	a asthma
r_3	b ulcer
d_1	b dummy ₁
r_4	e hernia
r_5	e flu
r_6	e' cancer
d_2	e' dummy ₂
$\bar{r}_3$	$\bar{b}$ ulcer
$\bar{d}_1$	$\bar{b}$ dummy ₁
m_2	x marker ₂
d_3	x dummy ₃

Storage servers

$$J^*$$

L.I	L.Attr	R.I	R.Attr
l_1	a Ann	a	flu
l_1	a Ann	a	asthma
l_2	b Beth	b	ulcer
l_2	b Beth	b	dummy ₁
$\bar{l}_2$	$\bar{b}$ Beth	$\bar{b}$	ulcer
$\bar{l}_2$	$\bar{b}$ Beth	$\bar{b}$	dummy ₁
m_1	x marker ₁	x	marker ₂
m_1	x marker ₁	x	dummy ₃

Client

$$R_l^*$$

I_k	L.Tuple _k
α	λ_1
α'	λ_1'
β	λ_2
β'	λ_2'
γ	λ_3
γ'	λ_3'
$\bar{\beta}$	λ_2
$\bar{\beta}'$	λ_2'
χ	μ_1

$$R_r^*$$

I_k	R.Tuple _k
α	ρ_1
α	ρ_2
β	ρ_3
β	δ_1
ε	ρ_4
ε	ρ_5
ε'	ρ_6
ε'	δ_2
$\bar{\varepsilon}$	$\bar{\rho}_3$
$\bar{\beta}$	δ_1
χ	μ_2
χ	δ_3

$$J_k^*$$

L.I _k	L.Tuple _k	R.I _k	R.Tuple _k
α	λ_1	α	ρ_1
α	λ_1	α	ρ_2
β	λ_2	β	ρ_3
β	λ_2	β	δ_1
$\bar{\beta}$	λ_2	$\bar{\beta}$	$\bar{\rho}_3$
$\bar{\beta}$	λ_2	$\bar{\beta}$	δ_1
χ	μ_1	χ	μ_2
χ	μ_1	χ	δ_3

Computational cloud

Join execution – Example

	R_l^*
	I Attr
l_1	a Ann
l_1'	a' Ann'
l_2	b Beth
l_2'	b' Beth'
l_3	c Cloe
l_3'	c' Cloe'
$\bar{l}_2$	$\bar{b}$ Beth
$\bar{l}_2'$	$\bar{b}'$ Beth'
m_1	x marker ₁

	R_r^*
	I Attr
r_1	a flu
r_2	a asthma
r_3	b ulcer
d_1	b dummy ₁
r_4	e hernia
r_5	e flu
r_6	e' cancer
d_2	e' dummy ₂
$\bar{r}_3$	$\bar{b}$ ulcer
$\bar{d}_1$	$\bar{b}$ dummy ₁
m_2	x marker ₂
d_3	x dummy ₃

Storage servers

	J^*
	L.I L.Attr R.I R.Attr
l_1	a Ann a flu
l_1'	a Ann a asthma
l_2	b Beth b ulcer

r_1
 r_2
 r_3

Client

	R_l^*
I_k	L.Tuple _k
α	λ_1
α'	λ_1'
β	λ_2
β'	λ_2'
γ	λ_3
γ'	λ_3'
$\bar{\beta}$	λ_2
$\bar{\beta}'$	λ_2'
χ	μ_1

	R_r^*
I_k	R.Tuple _k
α	ρ_1
α	ρ_2
β	ρ_3
β	δ_1
ε	ρ_4
ε	ρ_5
ε'	ρ_6
ε'	δ_2
$\bar{\varepsilon}$	$\bar{\rho}_3$
$\bar{\beta}$	δ_1
χ	μ_2
χ	δ_3

J_k^*

L.I. _k	L.Tuple _k	R.I. _k	R.Tuple _k
α	λ_1	α	ρ_1
α	λ_1	α	ρ_2
β	λ_2	β	ρ_3
β	λ_2	β	δ_1
$\bar{\beta}$	λ_2	$\bar{\beta}$	$\bar{\rho}_3$
$\bar{\beta}$	λ_2	$\bar{\beta}$	δ_1
χ	μ_1	χ	μ_2
χ	μ_1	χ	δ_3

Computational cloud

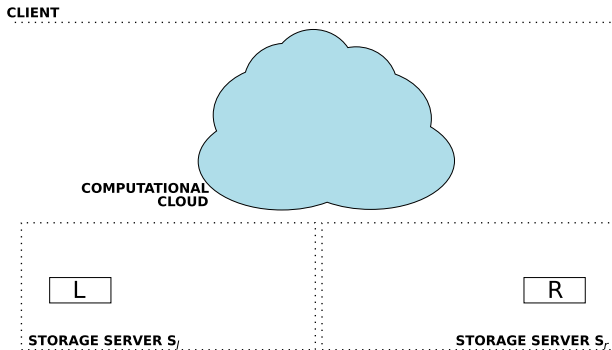
Markers and twins: Integrity guarantees

- The guarantee offered by markers and twins can be measured as the probability of the computational cloud to go undetected when omitting tuples
- Markers and twins offer complementary protection:
 - Twins are twice as effective as markers, but lose their effectiveness when the computational cloud omits a large fraction of tuples (extreme case: all tuples omitted)
 - Markers allow detecting extreme behavior (all tuples omitted) and provide effective when the computational cloud omits a large fraction of tuples

Semi-join execution strategy – 1

- Salts and buckets introduce computation and communication overhead
- Semi-join execution strategy [DFJLPS-14b]
 - protect the join profile without the need of introducing salts and buckets
 - support one-to-one, one-to-many, many-to-many joins and join sequences

Semi-join execution strategy – 2



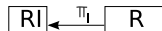
Semi-join execution strategy – 2

CLIENT

COMPUTATIONAL
CLOUD



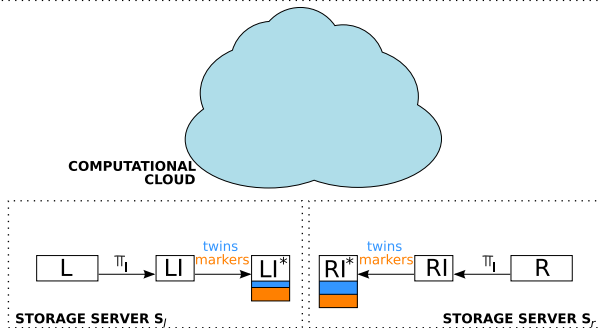
STORAGE SERVER S_i



STORAGE SERVER S_j

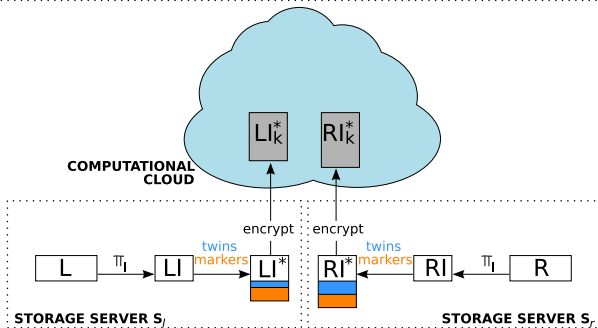
Semi-join execution strategy – 2

CLIENT



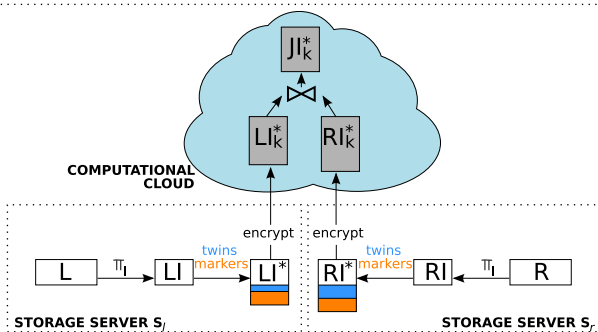
Semi-join execution strategy – 2

CLIENT

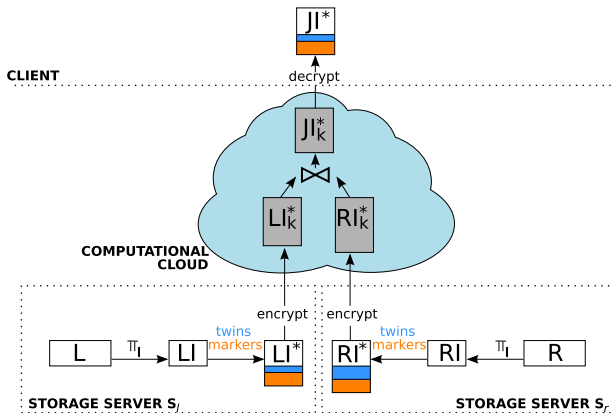


Semi-join execution strategy – 2

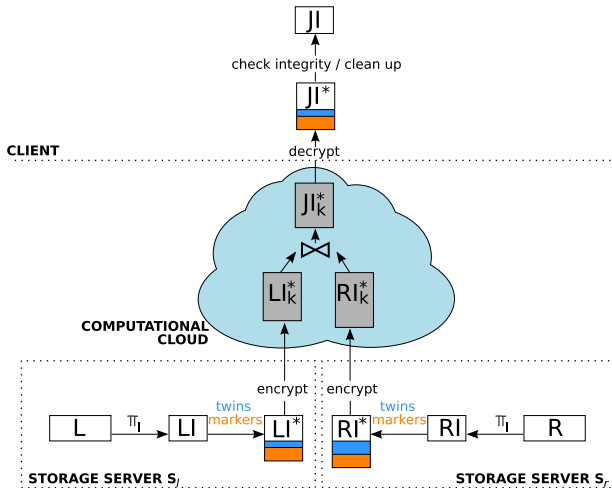
CLIENT



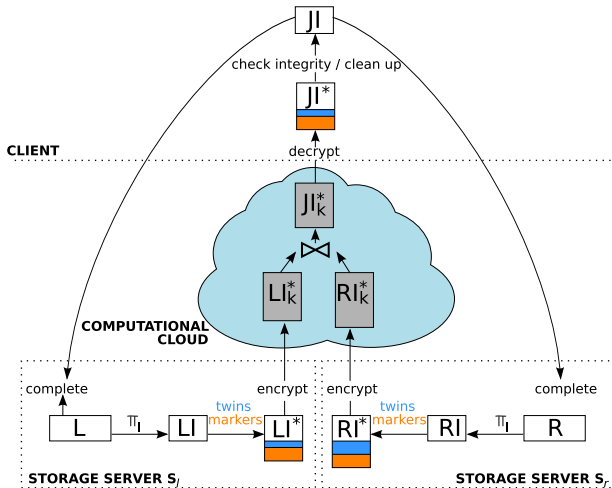
Semi-join execution strategy – 2



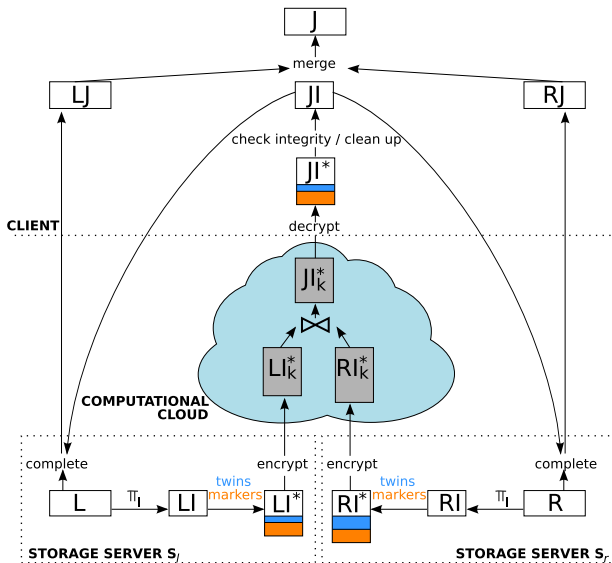
Semi-join execution strategy – 2



Semi-join execution strategy – 2



Semi-join execution strategy – 2



Distributed computational cloud

Some computational cloud scenarios support the processing of a vast amount of data in parallel on a large number of nodes (e.g., MapReduce)

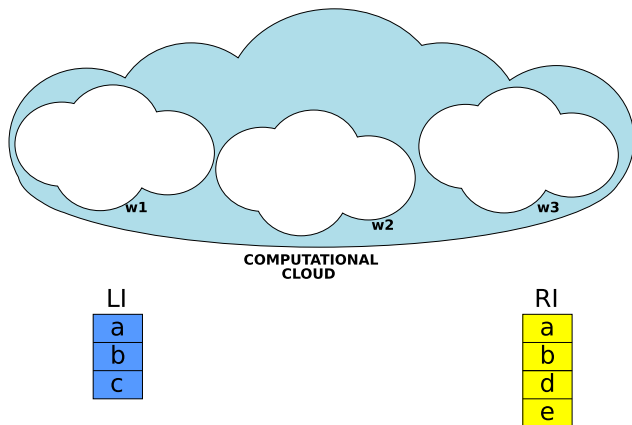
- need to reason about different nodes involved in the enforcement of integrity controls and ensure
 - control is well distributed among different nodes
 - ability to recognize misbehaving nodes (accountability)

Computational cloud working: MapReduce

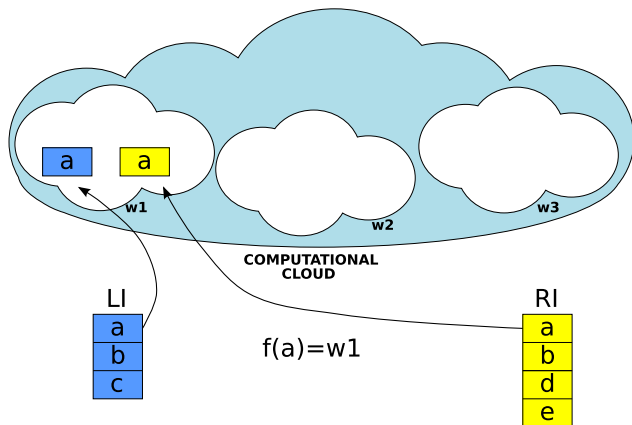
A **MapReduce** framework supports execution of tasks over large amount of data in parallel by multiple nodes (**worker**), coordinated by a **manager**

- A user-defined **map function** translates the input (tuples to be joined) in a set of pairs $\langle key, value \rangle$
- An **assignment function** f assigns pairs $\langle key, value \rangle$ to workers $\implies$ all pairs with the same key go to the same worker $w = f(key)$
- A user-defined **reduce function** (join operation) is executed by each worker, and the result is then combined by the manager

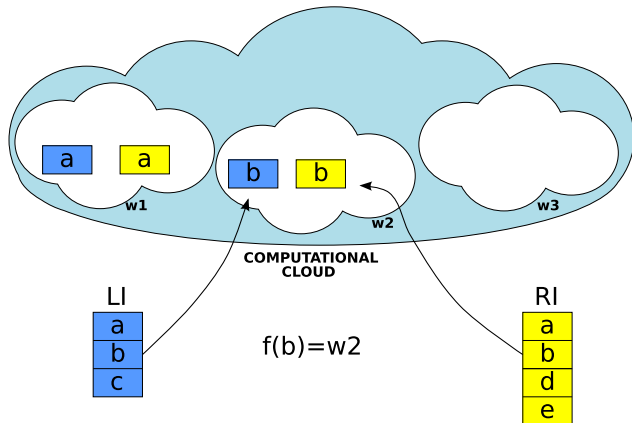
Computational cloud working: MapReduce – example



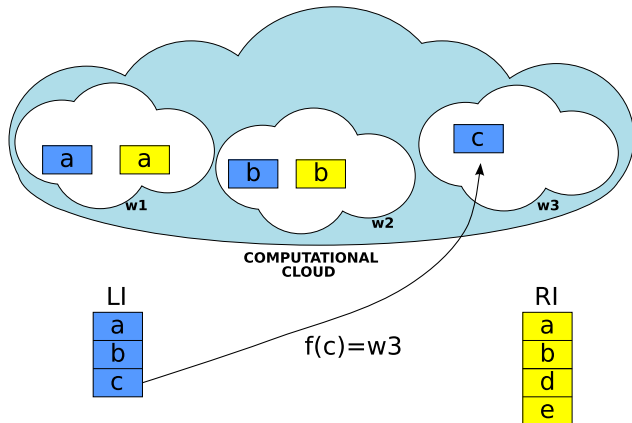
Computational cloud working: MapReduce – example



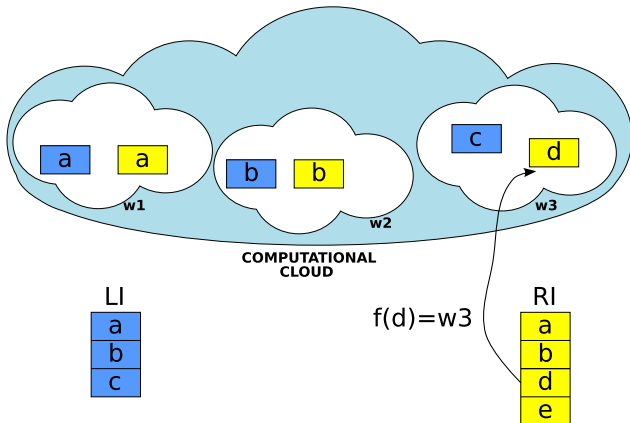
Computational cloud working: MapReduce – example



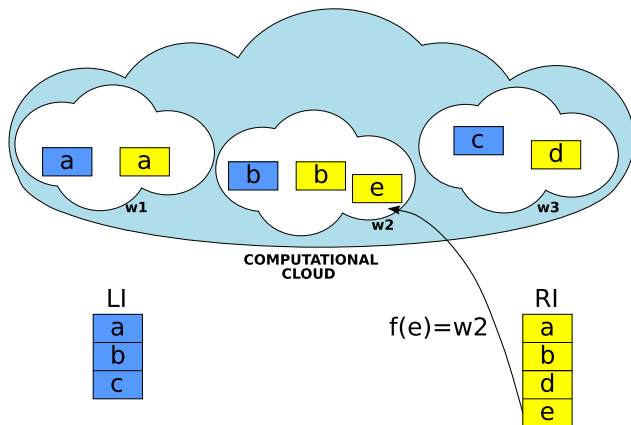
Computational cloud working: MapReduce – example



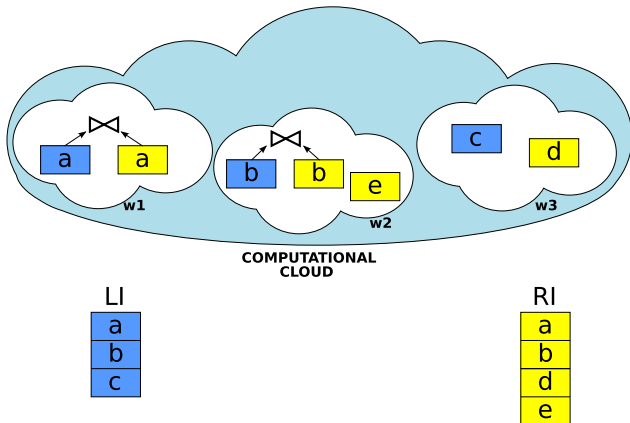
Computational cloud working: MapReduce – example



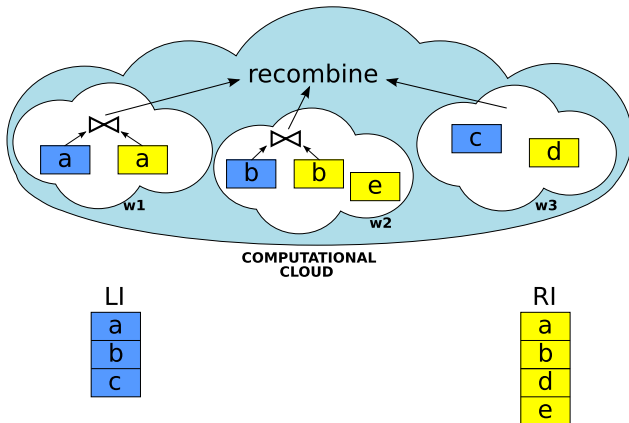
Computational cloud working: MapReduce – example



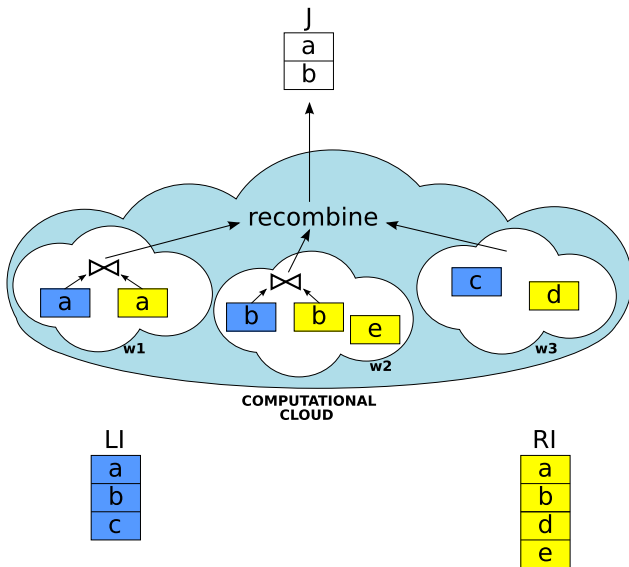
Computational cloud working: MapReduce – example



Computational cloud working: MapReduce – example



Computational cloud working: MapReduce – example



On-the-fly encryption

Encryption is applied to the join attribute of the relations involved in the join before they are passed to the computational cloud

- Every storage server **encrypts** its relation B obtaining $B^k(I_k)$, with I the join attribute of B
 - for each **distinct** $t[I]$ in B , there is τ in $B^k : \tau[I_k] = E_k(t[I])$
 - E is a **symmetric** encryption function with key k
 - k is defined by the client and **changes** at every query
- **Encrypted values** are translated into pairs $\langle \text{key}, \text{value} \rangle$ of the form $\langle \tau[I^k], B^k \rangle$
 - tuples with the **same values for the join attribute** are assigned to the **same worker**
 - no tuple is missed from the join due to an improper allocation

Markers and MapReduce (1)

- Markers should be properly distributed among all l workers (to distribute control)
- Marker distribution strategy $\langle N, N_{min}, N_{max} \rangle$ with N the total number of markers, N_{min}/N_{max} the minimum/maximum number of markers per worker
 - random $\langle N, 0, N \rangle$: no condition on the distribution of markers to workers
 - at-least- n $\langle N, n, n + (N - n \cdot l) \rangle$: every worker must receive at least n markers ($n \leq \lfloor N/l \rfloor$)
 - perfect balance $\langle N, \lfloor N/l \rfloor, \lceil N/l \rceil \rangle$: markers should be distributed evenly (the number of markers at any pair of workers can differ by at most one)

Markers and MapReduce (2)

All storage servers generate markers with function μ set by the client

Generate_Markers(N, N_{min}, N_{max})

```
1:  $spare := N - (N_{min} * l)$  /* spare markers */
2: repeat
3:   generate a new marker  $m$  via function  $\mu$ 
4:   let  $w$  be  $f(E_k(m))$  and  $n_w$  be the number of markers already assigned to it
5:   if ( $n_w < N_{min}$ ) or ( $n_w < N_{max}$  and  $spare > 0$ )
6:     then retain  $m$ 
7:        $n_w := n_w + 1$ 
8:       if  $n_w > N_{min}$  then  $spare := spare - 1$ 
9:     else discard  $m$ 
10: until  $N$  markers have been allocated
```

Markers and MapReduce (3)

- Every storage server generates the same set of markers
 - each server produces the same sequence of markers
 - allocation of markers to workers is deterministic
- The generated markers are correct:
 - for each worker w : $N_{min} \leq n_w \leq N_{max}$
 - the total number of assigned markers is N

Twins and MapReduce (1)

- Twins also should be properly distributed on the different workers
- Controlling twin generation like for markers is not possible
⇒ twins depend on the join attribute values at each server
 - each server can twin different tuples depending on its instance
 - each server can observe a different number of twins for a worker
 - servers cannot coordinate to regulate twin distributions
- Twin separation: a twin cannot be assigned to the same worker as its original tuple
 - property on which all servers have the same visibility
 - two-man-rule: a worker missing t would be exposed by the presence of $\bar{t}$ on a different worker (and viceversa)

Twins and MapReduce (2)

Storage servers twin tuples based on condition C_{twin} and a salt generating function σ set by the client

Generate_Twins(B, C_{twin})

```
1: for each  $t$  in  $B$  satisfying condition  $C_{\text{twin}}$  do  
2:   let  $w$  be  $f(E_k(t[I]))$   
3:   repeat  
4:     generate salt  $s$  via function  $\sigma$   
5:      $\bar{t} := t$   
6:     let  $\bar{w}$  be  $f(E_k(\bar{t}[I] \oplus s))$   
7:   until  $\bar{w} \neq w$ 
```

Twins and MapReduce (2)

Storage servers twin tuples based on condition C_{twin} and a salt generating function σ set by the client

Generate_Twins(B, C_{twin})

```
1: for each  $t$  in  $B$  satisfying condition  $C_{\text{twin}}$  do  
2:   let  $w$  be  $f(E_k(t[I]))$   
3:   repeat  
4:     generate salt  $s$  via function  $\sigma$   
5:      $\bar{t} := t$   
6:     let  $\bar{w}$  be  $f(E_k(\bar{t}[I] \oplus s))$   
7:   until  $\bar{w} \neq w$ 
```

Twin pairs are **guaranteed** to participate in the join result

- all servers generate twins with the same generation function
- allocation of twins to workers is **deterministic**

Join overall execution – example

CLIENT

COMPUTATIONAL
CLOUD

I	Premium
a	200
b	400
c	300

STORAGE SERVER S₁

I	Disease	Physician
b	flu	White
a	asthma	Warren
b	flu	Warren
e	stroke	Welsh
d	gastritis	Welsh

STORAGE SERVER S₂

Join overall execution – example

CLIENT

COMPUTATIONAL
CLOUD

I	Premium
a	200
b	400
c	300

 $\xrightarrow{\pi_I}$

I
a
b
c

STORAGE SERVER S_i

I	Disease	Physician
b	flu	White
a	asthma	Warren
b	flu	Warren
e	stroke	Welsh
d	gastritis	Welsh

 $\xleftarrow{\pi_I}$

I
a
b
d
e

STORAGE SERVER S_j

Join overall execution – example

CLIENT

COMPUTATIONAL
CLOUD

I	Premium
a	200
b	400
c	300

Π_1

I
a
b
c

twins
markers

I
a
b
c
a
c
m

STORAGE SERVER S_1

I
a
b
d
e
a
m

twins
markers

I
a
b
d
e

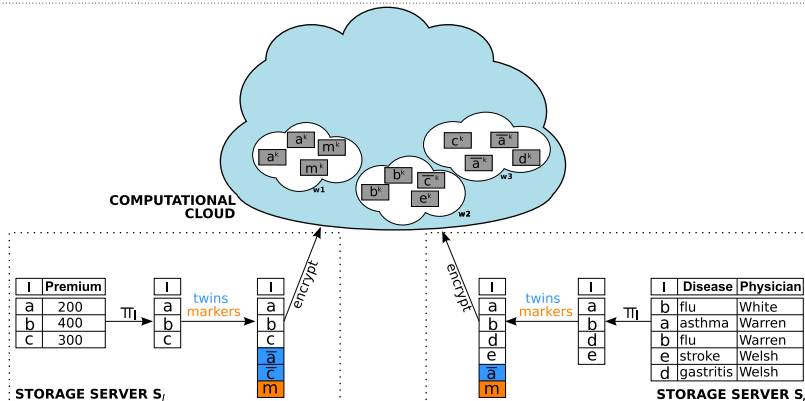
Π_1

I	Disease	Physician
b	flu	White
a	asthma	Warren
b	flu	Warren
e	stroke	Welsh
d	gastritis	Welsh

STORAGE SERVER S_2

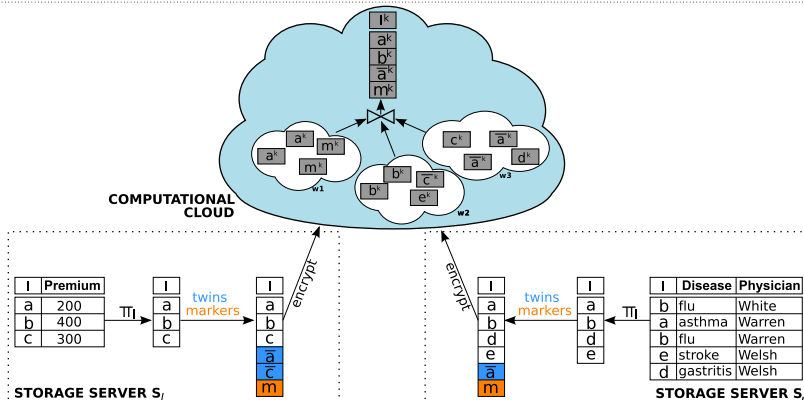
Join overall execution – example

CLIENT

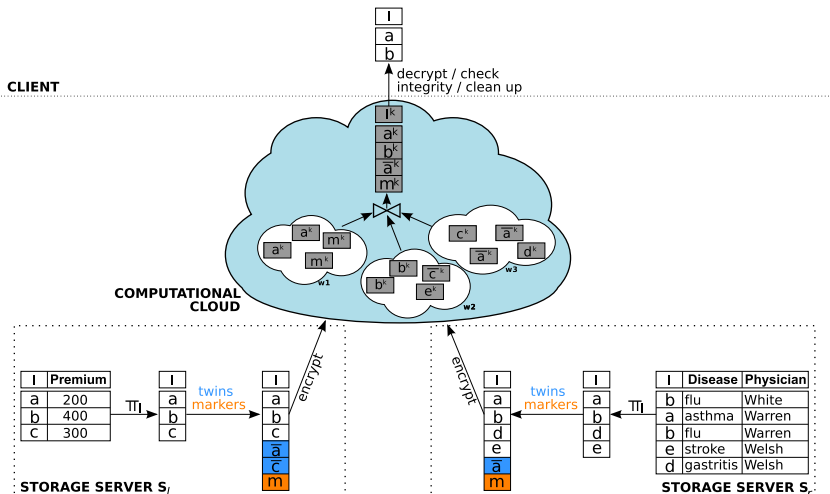


Join overall execution – example

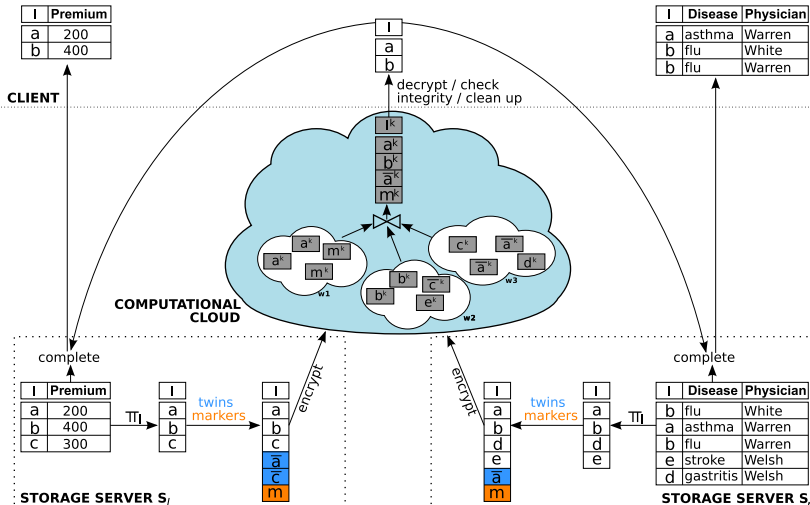
CLIENT



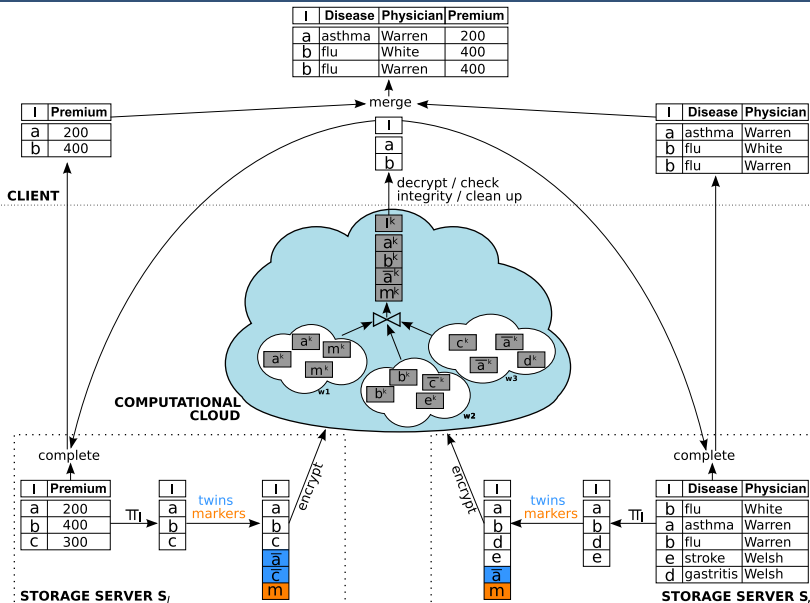
Join overall execution – example



Join overall execution – example



Join overall execution – example



Variations/open issues ...

- Execution of a join as a semi-join to support n:m joins and protect join profile [DFJPS-14]
- Application of the techniques to only a portion of the data (verification object) [DFJPS-14]
- Consideration of different trust levels
- Removal of trust assumptions in the storage servers

References – 1

- [CKGS-98] B. Chor, E. Kushilevitz, O. Goldreich, M. Sudan, “Private Information Retrieval,” in *JACM*, vol. 45, no. 6, 1998.
- [DFJLPS-14] S. De Capitani di Vimercati, S. Foresti, S. Jajodia, G. Livraga, S. Paraboschi, P. Samarati, “Integrity for Distributed Queries,” in *Proc. of CNS*, San Francisco, CA, USA, October 2014.
- [DFJPS-14] S. De Capitani di Vimercati, S. Foresti, S. Jajodia, S. Paraboschi, P. Samarati, “Optimizing Integrity Checks for Join Queries in the Cloud,” in *Proc. of DBSec*, Vienna, Austria, July 2014.
- [DFJPS-13] S. De Capitani di Vimercati, S. Foresti, S. Jajodia, S. Paraboschi, P. Samarati, “Integrity for Join Queries in the Cloud,” in *IEEE Transactions on Cloud Computing (TCC)*, vol. 1, n. 2, July-December 2013, pp. 187-200.
- [DFPPS-11a] S. De Capitani di Vimercati, S. Foresti, S. Paraboschi, G. Pelosi, P. Samarati, “Efficient and Private Access to Outsourced Data,” in *Proc. of ICDCS*, Minneapolis, MN, USA, June 2011.
- [DFPPS-11b] S. De Capitani di Vimercati, S. Foresti, S. Paraboschi, G. Pelosi, P. Samarati, “Supporting Concurrency in Private Data Outsourcing,” in *Proc. of ESORICS*, Leuven, Belgium, September 2011.

References – 2

- [DFPPS-13] S. De Capitani di Vimercati, S. Foresti, S. Paraboschi, G. Pelosi, P. Samarati, “Distributed Shuffling for Preserving Access Confidentiality,” in *Proc. of ESORICS*, Egham, U.K., September 2013.
- [DGMS-00] P.T. Devanbu, M. Gertz, C.U. Martel, S.G. Stubblebine, “Authentic third-party data publication,” in *Proc. of DBSec*, Schoorl, The Netherlands, August 2000.
- [GT-00] M. Goodrich, R. Tamassia, “Efficient Authenticated Dictionaries with Skip Lists and Commutative Hashing’,” in Technical Report, Johns Hopkins Information, 2000.
- [LC-04] P. Lin, K.S. Candan, “Hiding Traversal of Tree Structured Data from Untrusted Data Stores,” in *Proc. of WOSIS*, Porto, Portugal, April 2004.
- [NT-05] M. Narasimha, G. Tsudik, “DSAC: Integrity for Outsourced Databases with Signature Aggregation and Chaining,” in *Proc. CIKM*, Bremen, Germany, October-November 2005.
- [PPP-10] B. Palazzi, M. Pizzonia, S. Pucacco, “Query Racing: Fast Completeness Certification of Query Results,” in *Proc. DBSEC*, Rome, Italy, June 2010.

References – 3

- [RFKSSvD-15] L. Ren, C. Fletcher, A. Kwon, E. Stefanov, E. Shi M. van Dijk, S. Devadas, “Constants Count: Practical Improvements to Oblivious RAM” in *Proc. of USENIX*, Washington, USA, August 2015.
- [S-05] R. Sion, “Query Execution Assurance for Outsourced Databases,” in *Proc. of VLDB*, Trondheim, Norway, August-September 2005.
- [SC-07] R. Sion, B. Carbunar, “On the Computational Practicality of Private Information Retrieval,” in *Proc. of NDSS*, San Diego, CA, USA, February/March 2007.
- [SVSFRYD-13] E. Stefanov, M. van Dijk, E. Shi, C. Fletcher, L. Ren, X. Yu, S. Devadas, “Path ORAM: An Extremely Simple Oblivious RAM Protocol,” in *Proc. of CCS*, Berlin, Germany, November 2013.
- [WSC-08] P. Williams, R. Sion, B. Carbunar, “Building Castles Out of Mud: Practical Access Pattern Privacy and Correctness on Untrusted Storage,” in *Proc. of CCS*, Alexandria, USA, October 2008.
- [WS-12] P. Williams, R. Sion, “Single Round Access Privacy on Outsourced Storage,” in *Proc. of CCS*, Raleigh, NC, USA, October 2012.
- [WYPY-08] H. Wang, J. Yin, C. Perng, P. Yu, “Dual Encryption for Query Integrity Assurance,” in *Proc. of CIKM*, Napa Valley, CA, USA, October 2008.

References – 4

- [XWYM-07] M. Xie, H. Wang, J. Yin, X. Meng, "Integrity Auditing of Outsourced Data," in *Proc. of VLDB*, Vienna, Austria, September 2007.